

COELHO REALTIME

A KUBERNETES-NATIVE MLOPS PLATFORM THAT
FUSES INCREMENTAL LEARNING WITH BATCH
INTELLIGENCE

*TRANSACTION FRAUD DETECTION
ESTIMATED TIME OF ARRIVAL
E-COMMERCE CUSTOMER INTERACTIONS*

*INCREMENTAL MACHINE LEARNING
BATCH MACHINE LEARNING
DELTA LAKE SQL*

RAFAEL COELHO

ABOUT THE PROJECT

COELHO RealTime is a demonstrative, Kubernetes-native MLOps (Machine Learning Operations) platform, built to demonstrate production-grade MLOps capabilities through simulated real-world data and scenarios.

It masterfully combines **Incremental Learning** with **Batch Intelligence** via a unique dual-pipeline, enabling both real-time model adaptation and robust batch training.

This platform addresses critical challenges in areas like **Transaction Fraud Detection**, **Estimated Time of Arrival**, and **E-commerce Customer Interactions**.

Orchestrated on K3d with end-to-end observability from **Prometheus**, **Grafana**, and **MLflow**, COELHO RealTime ensures the demonstration of accurate, transparent, and continuously optimized machine learning models for dynamic operational demands.



PLATFORM ARCHITECTURE

01.

INCREMENTAL & BATCH MACHINE LEARNING

The platform combines two complementary machine learning paradigms into a unified dual-pipeline architecture.

The Machine Learning Layer provides real-time model adaptation through River's incremental learning and robust batch training via Scikit-Learn and CatBoost.

The Data & Streaming Layer feeds these models through Apache Kafka event streaming and Apache Spark Structured Streaming, persisting all data as Delta Lake tables on MinIO.

The Application Services Layer exposes model training, predictions, and diagnostics through FastAPI, with Redis caching live models and MLflow tracking every experiment.

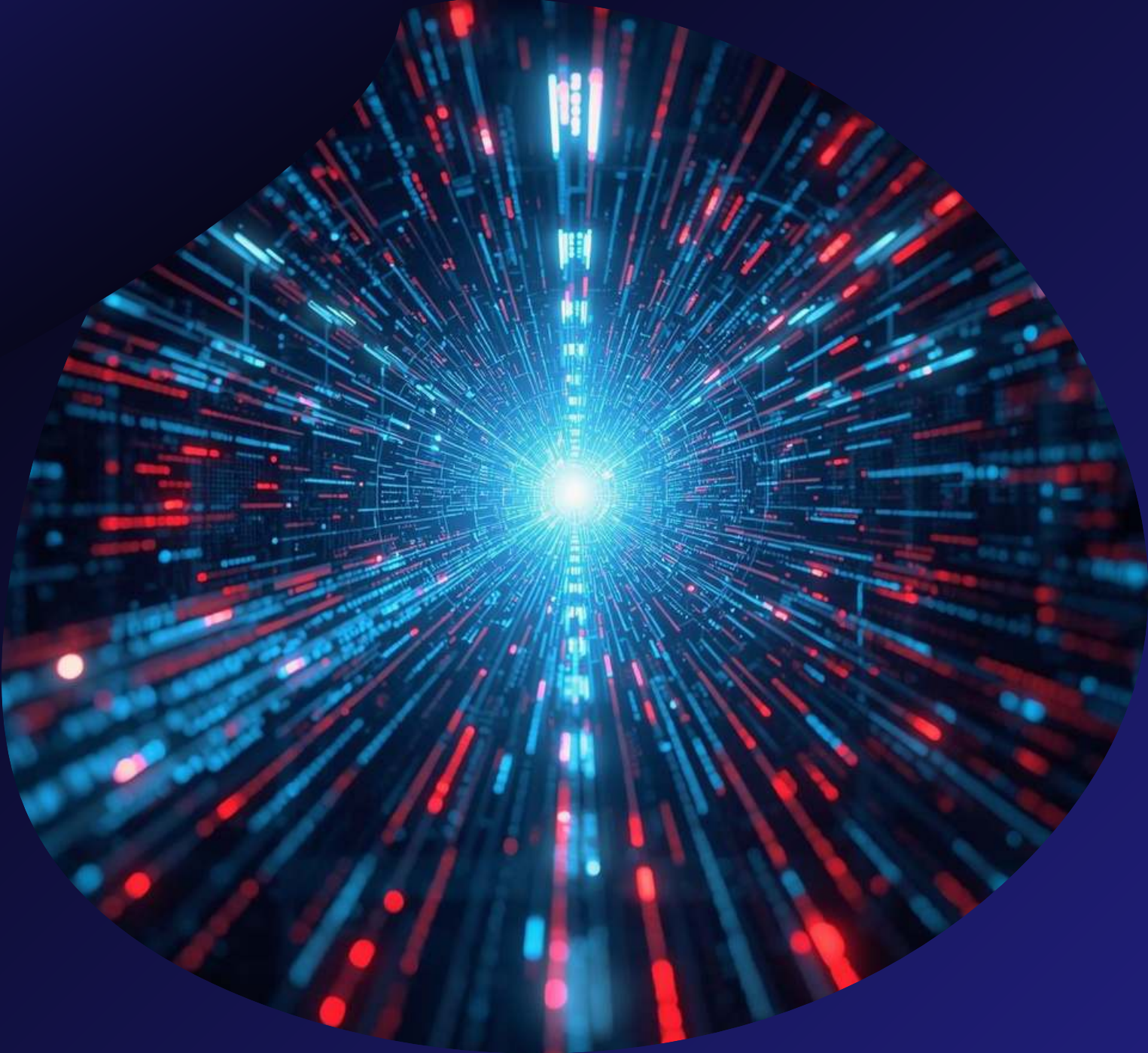
02.

INFRASTRUCTURE AS CODE & MICROSERVICES

The entire platform — 15 containerized microservices — is provisioned declaratively through Terraform and orchestrated on Kubernetes via K3d.

The Container Orchestration Layer establishes the cluster foundation, while the GitOps & CI/CD Layer automates the complete delivery pipeline from GitLab CI builds through ArgoCD deployments with automated image updates.

The Observability & Monitoring Layer ensures operational transparency through Prometheus metrics collection, Grafana visualization, and Alertmanager with dependency-aware inhibition rules, all backed by PostgreSQL metadata persistence.



01.

INCREMENTAL & BATCH MACHINE LEARNING

The platform combines two complementary machine learning paradigms into a unified dual-pipeline architecture.

River's incremental learning enables models to adapt in real-time as Kafka streams arrive, while Scikit-Learn and CatBoost deliver robust batch training on historical Delta Lake datasets.

Both pipelines share the same data, the same tracking (MLflow), and the same serving layer (FastAPI) — ensuring every model is continuously optimized whether learning from one event or one million rows.

**COELHO
REALTIME**

MAIN APPLICATIONS



Transaction Fraud Detection

This demonstrative application showcases an adaptive MLOps platform that detects and mitigates fraud in real-time, simulating real-world scenarios to protect assets and maintain trust.



Estimated Time of Arrival

This demonstrative application, simulating real-world data, provides highly accurate and dynamic ETAs by leveraging real-time data and predictive analytics, optimizing logistics and boosting customer satisfaction.



E-Commerce Customer Interactions

A demonstrative application, this project simulates real-world e-commerce scenarios to showcase how personalized recommendations and dynamic content can transform customer journeys, driving engagement, fostering loyalty, and increasing conversion rates.

MAIN APPLICATIONS



TRANSACTION FRAUD DETECTION

- **Dual-Pipeline Model Deployment:** Showcases combining Incremental Learning (River) for real-time fraud pattern adaptation with Batch Intelligence (Scikit-Learn and CatBoost) for robust historical analysis.
 - **Kafka-driven Data Ingestion:** Demonstrates real-time transaction data streaming using Apache Kafka producers.
 - **FastAPI Prediction Service:** Exposes TFD model predictions via FastAPI endpoints for low-latency inference.
 - **Comprehensive Observability:** Fraud detection metrics are monitored with Prometheus, visualized in Grafana, and model experiments are tracked in MLflow.
-



ESTIMATED TIME OF ARRIVAL

- **Dual-Pipeline Model Deployment:** Utilizes Incremental Learning (River) for dynamic ETA adjustments based on live changes, and Batch Intelligence (Scikit-Learn and CatBoost) for foundational ETA model training.
 - **Kafka-driven Data Streams:** Simulates and streams real-time data (e.g., traffic, weather) influencing ETAs via Apache Kafka.
 - **FastAPI Prediction Service:** Provides ETA predictions through FastAPI endpoints.
 - **Comprehensive Observability:** ETA model performance and system health are monitored with Prometheus, visualized in Grafana, and experiments are managed in MLflow.
-



E-COMMERCE CUSTOMER INTERACTIONS

- **Dual-Pipeline Model Deployment:** Combines Incremental Learning (River) for adapting to changing customer preferences and behavior with Batch Intelligence (Scikit-Learn) for comprehensive historical behavioral analysis.
- **Kafka-driven User Data:** Simulates and streams real-time e-commerce interaction data via Apache Kafka producers.
- **FastAPI Prediction Service:** Serves ECCL-related predictions through FastAPI.
- **Comprehensive Observability:** Customer interaction model metrics are tracked using Prometheus, visualized in Grafana, and experiment management with MLflow.

FOUNDATIONAL MLOPS COMPONENTS



Incremental Machine Learning

Leveraging frameworks like River, this component enables models (TFD, ETA, ECCI) to continuously learn from real-time data streams. This ensures dynamic adaptation and sustained accuracy, effectively combating concept drift in evolving environments.



Batch Machine Learning

This component establishes robust model baselines and deep insights by training on extensive historical datasets using Apache Spark, Scikit-Learn and CatBoost. It provides a powerful, scalable foundation for all TFD, ETA, and ECCI applications.



Delta Lake SQL

Powered by Delta Lake and accessible via SQL on DuckDB, this foundational layer serves as the central, reliable, and consistent data repository. It seamlessly integrates and supports both high-throughput streaming (from Kafka) and large-scale batch processing for all project data.

INFRASTRUCTURE AS CODE & MICROSERVICES

- 
- 1 - CONTAINER ORCHESTRATION LAYER
 - 2 - GITOPS & CI/CD LAYER
 - 3 - DATA & STREAMING LAYER
 - 4 - APPLICATION SERVICES LAYER
 - 5 - OBSERVABILITY & MONITORING LAYER

1 - CONTAINER ORCHESTRATION LAYER



TERRAFORM

- Provisions K3d cluster with 1 server + 3 agent nodes through declarative HCL configuration
 - Manages Kubernetes, Helm, and K3d providers in a single state file, ensuring consistent cluster setup
 - Configures host volume mounts for persistent storage, enabling data survival across cluster rebuilds
 - **Why Terraform:** Eliminates OS-specific scripts, providing an agnostic Infrastructure-as-Code approach that works across Linux, Windows, and macOS without requiring translations
-



K3D

- Runs lightweight K3s distributions inside Docker containers, replicating production Kubernetes locally
 - Provides built-in container registry for local image distribution during CI/CD workflows
 - Supports multi-node clusters (1 server + 3 agents) with configurable NodePort mappings for service access
 - **Why K3d:** Enables a production-like Kubernetes environment on a single development machine with minimal resource overhead, bridging local development and cloud deployment
-



KUBERNETES

- Orchestrates all 15 containerized services with declarative manifests, managing scheduling, networking, and lifecycle
 - Handles service discovery via ClusterIP and external access via NodePort, connecting frontend to backend seamlessly
 - Enforces liveness/readiness probes, resource quotas, and automatic pod restart for self-healing operations
 - **Why Kubernetes:** The industry-standard orchestration platform that provides the reliability, scalability, and operational patterns required for a production-grade microservice architecture
-



DOCKER

- Containerizes all custom services (FastAPI, SvelteKit, Kafka Producers) with optimized single-stage Dockerfiles
 - Provides BuildKit-accelerated layer caching for rapid image rebuilds during development
 - Serves as the runtime foundation for K3d's Kubernetes nodes, running the entire cluster in containers
 - **Why Docker:** Standardizes every service into portable, reproducible containers — a prerequisite for Kubernetes orchestration and consistent behavior across development and production
-

2 - GITOPS & CI/CD LAYER



GITLAB

- Ships a CI/CD-ready .gitlab-ci.yml that builds three Docker images (FastAPI, SvelteKit, Kafka Producers) via Docker-in-Docker, pluggable into any external GitLab instance
 - Tags every image with commit SHA and latest, pushing to the K3d local registry for full build traceability
 - Triggers automated builds on every master branch commit, bridging code changes to the deployment pipeline within seconds
 - **Why GitLab:** Portable pipeline definitions that plug into any GitLab instance, keeping the build-to-registry workflow environment-agnostic
-



ARGOCD

- Connects to an external ArgoCD via a pre-configured Application manifest declaring the complete Helm chart deployment for the coelho-realtime namespace
 - Auto-syncs cluster state to match Git with automated pruning, self-healing, and rollback across all 15 microservices
 - Paired with ArgoCD Image Updater, which watches the K3d registry for new images and commits updated tags back to Git
 - **Why ArgoCD:** Ready-to-apply GitOps manifests that transform any external ArgoCD into a fully declarative, automated deployment pipeline
-



SKAFFOLD

- Orchestrates local development with hot-reload file sync for Python and TypeScript directly into running pods
 - Manages port forwarding for 13 services simultaneously, exposing the complete platform on localhost
 - Intelligent caching and selective rebuilds based on file change detection
 - **Why Skaffold:** Eliminates the rebuild-redeploy cycle, enabling sub-second code changes in production-identical container environments
-



HELM

- Packages all 15 microservices into a single umbrella chart with seven subchart dependencies (MLflow, Redis, MinIO, PostgreSQL, kube-prometheus-stack, Kafka, Spark)
 - Declarative values.yaml centralizes configuration for every service: resource limits, ports, environment variables, and feature toggles
 - Conditional dependency enablement allows selective service deployment via simple boolean flags
 - **Why Helm:** The standard Kubernetes package manager that turns the entire platform into a single, versioned, reproducible deployment unit
-

3 - DATA & STREAMING LAYER



APACHE KAFKA

- Central event backbone in KRaft mode (no ZooKeeper) across three topics: transaction_fraud_detection, estimated_time_of_arrival, and e_commerce_customer_interactions
 - Fed by Python 3.13 producers generating realistic synthetic data (fraud patterns, geospatial deliveries, behavioral signals)
 - Auto-creates topics with 3 partitions for parallel consumption by River incremental learners and Spark batch processors
 - **Why Kafka:** Decouples data generation from consumption, letting the same stream feed both real-time and batch ML simultaneously
-



APACHE SPARK

- Runs three Structured Streaming jobs (kafka_to_delta_tfd.py, kafka_to_delta_eta.py, kafka_to_delta_ecommerce.py) consuming Kafka topics into Delta Lake tables
 - Configured with Delta Lake 4.0.0 JARs and Hadoop-AWS connector for native MinIO S3 compatibility
 - Deployed via Kubernetes ConfigMaps managed as part of the Helm chart lifecycle
 - **Why Spark:** Transforms raw Kafka streams into queryable Delta Lake tables, powering batch ML training with Scikit-Learn and CatBoost
-



DELTA LAKE

- Stores all streaming data as versioned, ACID-compliant tables on MinIO, supporting both streaming writes and batch reads
 - Enables time-travel queries and schema enforcement for data consistency across ML workloads
 - Queryable via DuckDB SQL through FastAPI endpoints, providing ad-hoc lakehouse access without Spark sessions
 - **Why Delta Lake:** Unifies streaming and batch data into a single lakehouse format with ACID transactions, eliminating separate data stores
-



MINIO

- Provides S3-compatible storage hosting two buckets: lakehouse for Delta Lake tables and mlflow-artifacts for experiment outputs
 - Persists data via hostPath volume mount (/data/minio), ensuring survival across pod restarts and cluster rebuilds
 - Exposes S3 API on port 9000, integrating seamlessly with Spark, MLflow, and any S3-compatible tooling
 - **Why MinIO:** Production-grade S3 storage locally, enabling cloud-identical patterns without external dependencies or costs
-

4 - APPLICATION SERVICES LAYER



FASTAPI

- Unified ML backend with three versioned API groups: /api/v1/incremental (River), /api/v1/batch (Scikit-Learn/CatBoost), /api/v1/sql (DuckDB queries)
 - Manages training subprocesses with orjson serialization and Redis-backed model caching for low-latency predictions
 - Instrumented with Prometheus metrics for request latency, throughput, and ML pipeline monitoring
 - **Why FastAPI:** Single entry point unifying incremental learning, batch training, and data querying into one coherent async API surface
-



SVELTEKIT

- Interactive frontend with SvelteKit 2.20, Vite 6.0, TailwindCSS 4.0, and Plotly.js for ML visualizations
 - Dedicated UI sections for TFD, ETA, and ECCL with training controls, predictions, metrics dashboards, and SQL editors
 - TypeScript strict mode with pre-bundled Vite caching for optimized container startup
 - **Why SvelteKit:** Reactive framework delivering instant UI updates for real-time ML metrics with minimal bundle overhead
-



REDIS

- Caches live incremental ML models in memory for sub-millisecond prediction serving with serialized River model state
 - Manages training status flags for real-time frontend progress display (LIVE/FINISHED) per application
 - ClusterIP service with configurable memory limits and eviction policies
 - **Why Redis:** In-memory speed bridging model training (seconds) and prediction latency (milliseconds) in the incremental pipeline
-



MLFLOW

- Tracks every experiment with parameters, metrics, and artifacts across both incremental and batch training pipelines
 - Metadata in PostgreSQL, artifacts on MinIO S3 — persistent history and full reproducibility
 - Tracking UI with run comparison and model registry integrated into the SvelteKit frontend
 - **Why MLflow:** Complete ML lifecycle management making every training run auditable, comparable, and reproducible
-

5 - OBSERVABILITY & MONITORING LAYER



PROMETHEUS

- Collects metrics from all 15 microservices via ServiceMonitors at 30-second intervals, centralizing operational and ML telemetry
 - Deploys via kube-prometheus-stack (v80.6.0) with 800+ alerting rules covering infrastructure health and resource utilization
 - Includes Node Exporter, kube-state-metrics, and Prometheus Operator for declarative monitoring
 - **Why Prometheus:** Cloud-native monitoring standard providing pull-based collection, PromQL querying, and alerting at scale
-



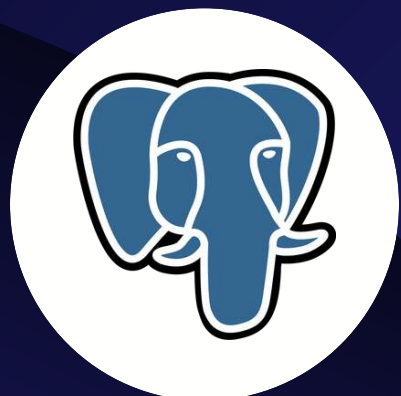
GRAFANA

- Custom dashboards for ML pipeline throughput, API latency, and infrastructure health visualization
 - Shares PostgreSQL backend with MLflow for unified metadata storage and reduced footprint
 - Dashboard-as-code provisioning through Helm configuration for version-controlled monitoring
 - **Why Grafana:** Transforms raw Prometheus metrics into actionable visual intelligence for rapid pipeline diagnosis
-



ALERTMANAGER

- Severity-based alert routing with dependency-aware inhibition rules preventing alert storms from cascading failures
 - Configured with multi-channel notification for ML pipeline failures and infrastructure degradation
 - Integrates with Prometheus rules for proactive incident detection before user impact
 - **Why Alertmanager:** Converts detected anomalies into actionable notifications, surfacing issues before manual discovery
-



POSTGRESQL

- Shared relational backend for MLflow experiment tracking and Grafana dashboard/user storage
 - Persists via hostPath mount (/data/postgresql) ensuring metadata survival across restarts and rebuilds
 - ClusterIP service with configurable resource limits and connection pooling
 - **Why PostgreSQL:** ACID-compliant relational storage for persistent, queryable metadata across ML tracking and monitoring
-

PYTHON ECOSYSTEM

1 - ML FRAMEWORKS



River



Scikit-Learn



CatBoost



Imbalanced-learn



PySpark



MLflow

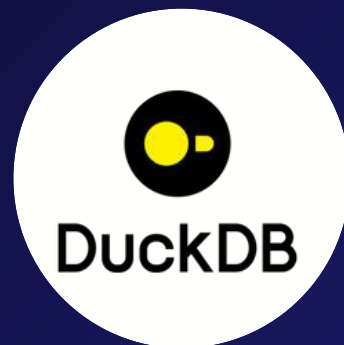
2 - DATA & STORAGE



Pandas



NumPy



DuckDB

3 - VISUALIZATION & DIAGNOSTICS



YellowBrick



Scikit-plot



UMAP

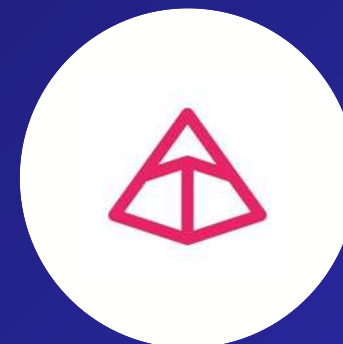


NLTK

4 - WEB FRAMEWORK



FastAPI

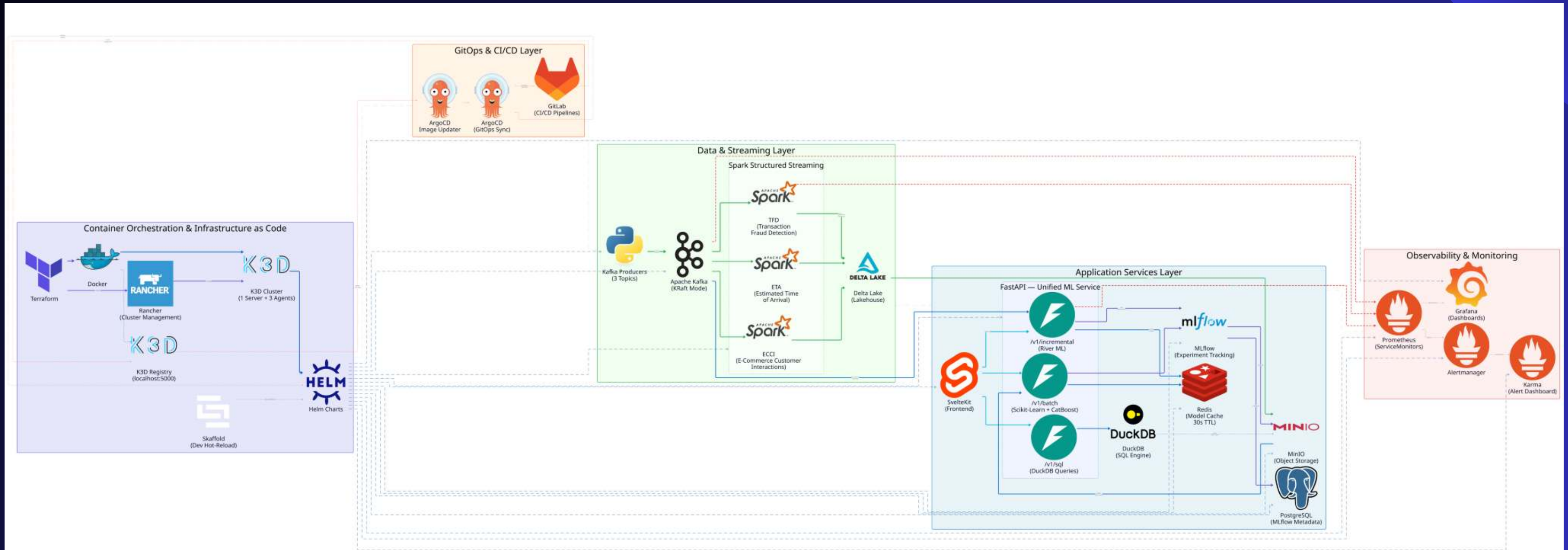


Pydantic



Redis

PLATFORM ARCHITECTURE



THE PLATFORM IN ACTION

A visual walkthrough of the COELHO RealTime platform — from its three demonstrative applications (Transaction Fraud Detection, Estimated Time of Arrival, and E-Commerce Customer Interactions) to its microservices infrastructure (Kafka, Spark, MLflow, Grafana, ArgoCD) — showcasing Incremental Machine Learning, Batch Machine Learning, and Delta Lake SQL capabilities running end-to-end on Kubernetes.



TRANSACTION FRAUD DETECTION

Incremental Machine Learning
Batch Machine Learning
Delta Lake SQL

- **Incremental ML model:** Adaptive Random Forest Classifier (River)
- **Batch ML model:** CatBoost Classifier

**COELHO
REALTIME**

TRANSACTION FRAUD DETECTION - INCREMENTAL MACHINE LEARNING

COELHO RealTime

COELHO REALTIME

Home

Applications

Services

Transaction Fraud Detection

Real-time fraud detection using streaming ML

Incremental ML

Batch ML

Delta Lake SQL

Real-time ML Training

Toggle to start processing live data

Adaptive Random Forest Classifier (River)

MLflow

Prediction

Metrics

Transaction Details

Predict

Randomize

Amount (1-6000)

3939.3

Account Age (0-1825)

2203

Currency

AUD

Date

01/25/2025

Time

2:33 PM

Merchant ID (1-200)

142

Category

travel

Type

deposit

Payment

paypal

Latitude (29.5-30.1)

29.950574

Longitude (-95.8--95)

-95.598269

Browser

Chrome

OS

Android

CVV

Provided

Billing

Address Match

Transaction ID: txn_dc9d7acb3067

User ID: user_8910

IP Address: 135.191.128.29

Prediction Result

MLflow FINISHED Run: 140e8c97 Started: 2026-01-25 02:37:42

HIGH RISK

Fraud Probability

77.8%

Classification

FRAUD

% Fraud

77.80%

Not Fraud

22.20%

1

Real-time ML training toggle

Activates or deactivates the continuous, real-time adaptation of the TFD model, enabling it to learn incrementally from incoming data streams.

2

MLflow model page - experiment run

Provides a summary of a specific incremental learning experiment for TFD on MLflow service page, detailing logged parameters, metrics, and associated artifacts within MLflow.

3

Prediction button

Initiates an immediate fraud prediction using the TFD model's current, incrementally updated state, demonstrating low-latency inference.

4

Prediction tab

Displays the interface where real-time fraud prediction outputs are presented.

5

MLflow experiment run informations

Offers status info (LIVE | FINISHED), MLflow run ID, and the datetime the last real-time ML model training was executed.

6

Prediction results

Presents the output of the TFD model's real-time fraud predictions

COELHO
REALTIME

TRANSACTION FRAUD DETECTION - INCREMENTAL MACHINE LEARNING

COELHO RealTime

COELHO REALTIME

HomeApplicationsServices

Transaction Fraud Detection

Real-time fraud detection using streaming ML

Incremental ML

Batch ML

Delta Lake SQL

Real-time ML Training

Toggle to start processing live data

Adaptive Random Forest Classifier (River)MLflow

Transaction Details

PredictRandomize

Amount (1-6000)

3939.3

Account Age (0-1825)

2203

Currency

AUD

Date

01/25/2026

Time

2:33 PM

Merchant ID (1-200)

142

Category

travel

Type

deposit

Payment

davpal

Latitude (29.5-30.1)

29.950574

Longitude (-95.8--95)

-95.598269

Browser

Chrome

OS

Android

CVV

Provided

Billing

Address Match

Transaction ID: txn_dc9d7acb3067

User ID: user_8910

IP Address: 135.191.128.29

PredictionMetrics

Classification Metrics

MLflowFINISHEDRun: 140e8c97Started: 2026-01-25 02:37:42

F-Beta0.5372

ROC AUC0.9927

Precision0.1884

Recall1.0000

Rolling ROC0.9928

F10.3171

Accuracy0.9729

Geo Mean0.9863

Cohen k0.3098

Jaccard0.1884

LogLoss0.2438

MCC0.428

Balanced Accuracy0.986

Confusion Matrix

Classification Report

1 Metrics tab

Presents the real-time metrics and incremental learning status of the TFD model.

- Metrics (River library)**
- Recall
 - Precision
 - F1
 - F-Beta
 - Accuracy
 - Balanced Accuracy
 - Matthews Correlation Coefficient
 - Geometric Mean
 - Cohen Kappa
 - Jaccard
 - ROC-AUC
 - Rolling ROC-AUC
 - Log Loss
 - Confusion Matrix
 - Classification Report

TRANSACTION FRAUD DETECTION - BATCH MACHINE LEARNING

The screenshot displays the COELHO REALTIME Transaction Fraud Detection interface. The top navigation bar includes the COELHO RealTime logo, the title 'COELHO REALTIME', and links for Home, Applications, and Services. The main header shows 'Transaction Fraud Detection' with the subtitle 'Real-time fraud detection using streaming ML'. Below this, there are tabs for 'Incremental ML', 'Batch ML' (which is selected), and 'Delta Lake SQL'. The interface is divided into several sections:

- Batch ML Training:** Located on the left, it shows 'Model trained and ready'. It includes a 'Data' section with 'Max Rows' set to 10000 and a 'rows' dropdown. A red box highlights the 'Train' button (1).
- Transaction Details:** Located below the training section, it contains a 'Predict' button (5) and a 'Randomize' button. Below these are various input fields for transaction details: Amount (1-6000), Account Age (0-1825), Currency, Date, Time, Merchant ID (1-200), Category, Type, Payment, Latitude (29.5-30.1), Longitude (-95.8-95), Browser, OS, CVV, and Billing. At the bottom, it shows Transaction ID, User ID, and IP Address.
- Prediction:** Located on the right, it shows a 'Prediction' tab (6) and a 'Metrics' tab. Below the tabs, it displays 'MLflow CatBoost Classifier Best Run' with run ID '1e49c74e', start time '2026-01-24 20:34:14', and samples '10,000'. A red box highlights the 'Prediction Result' section (8), which includes a 'MEDIUM RISK' gauge showing '56.5%' fraud probability. Below the gauge, it shows 'Classification FRAUD' (56.51% Fraud) and 'Not Fraud' (43.49% Not Fraud).

Numbered callouts (1-8) are placed over the interface to highlight specific features and actions.

1 Train button

Initiates the batch machine learning training process for the TFD model.

2 MLflow run ID - ML model

Dropdown displaying the MLflow run IDs associated with batch-trained TFD model the user can pick after training new Batch ML models.

3 MLflow run ID page

Navigates to or provides detailed information within MLflow about a particular experiment run.

4 Batch ML Training options

Provides the user the options of training new Batch ML models in real-time per maximum number of rows or per percentage of rows from the original TFD dataset.

5 Prediction button

Triggers a prediction using the currently selected or trained batch ML model.

6 Prediction tab

Displays the interface or section where batch prediction results for the TFD model are presented.

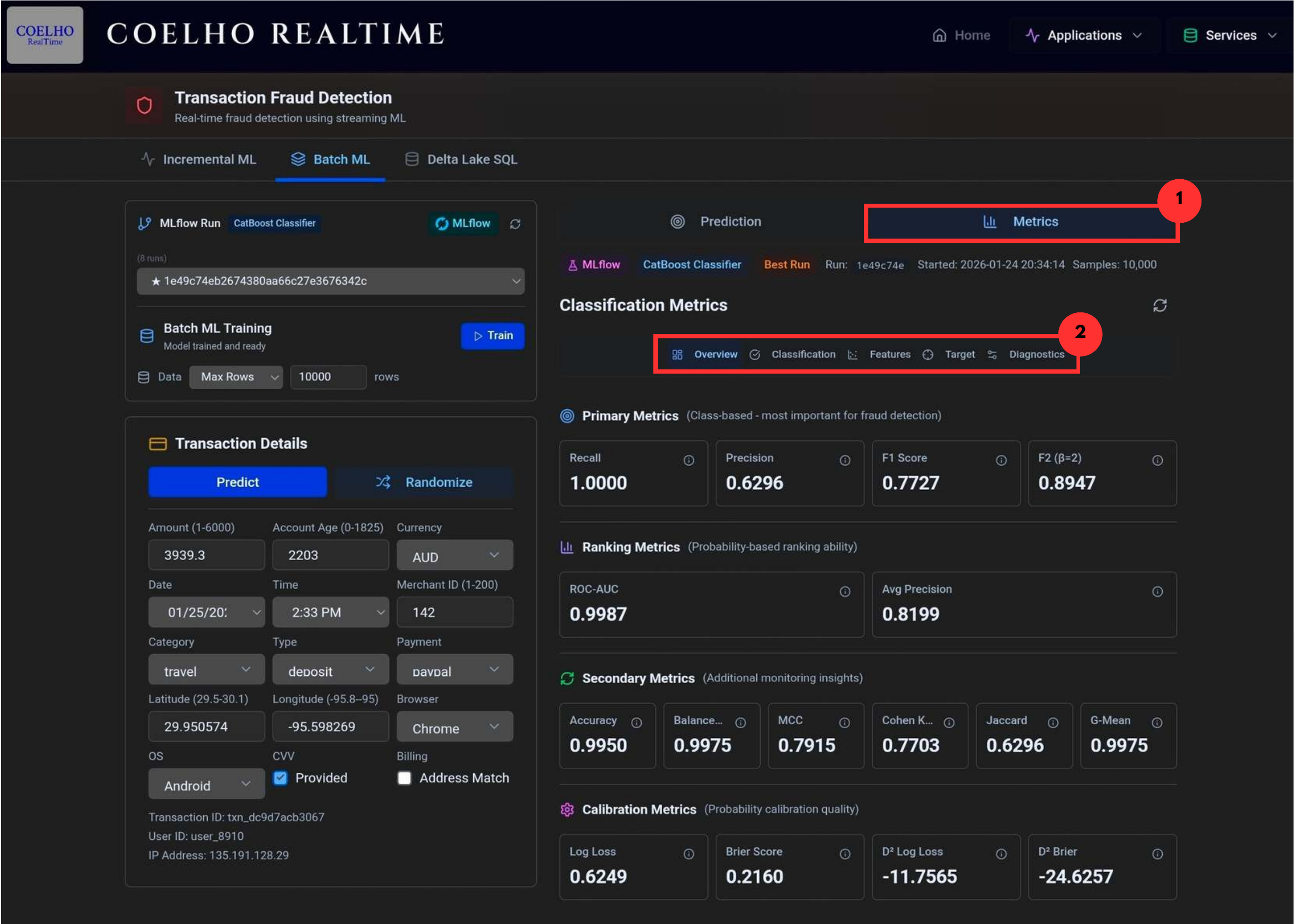
7 MLflow experiment run informations

Offers detailed status information, run ID, and other metadata for an MLflow experiment run.

8 Prediction results

Presents the output of the TFD model's real-time fraud predictions.

TRANSACTION FRAUD DETECTION - BATCH MACHINE LEARNING



1 Metrics tab

Presents various metrics of the TFD model. Includes some tabs with a lot of visual metrics from Scikit-Learn, Scikit-Plot and YellowBrick libraries.

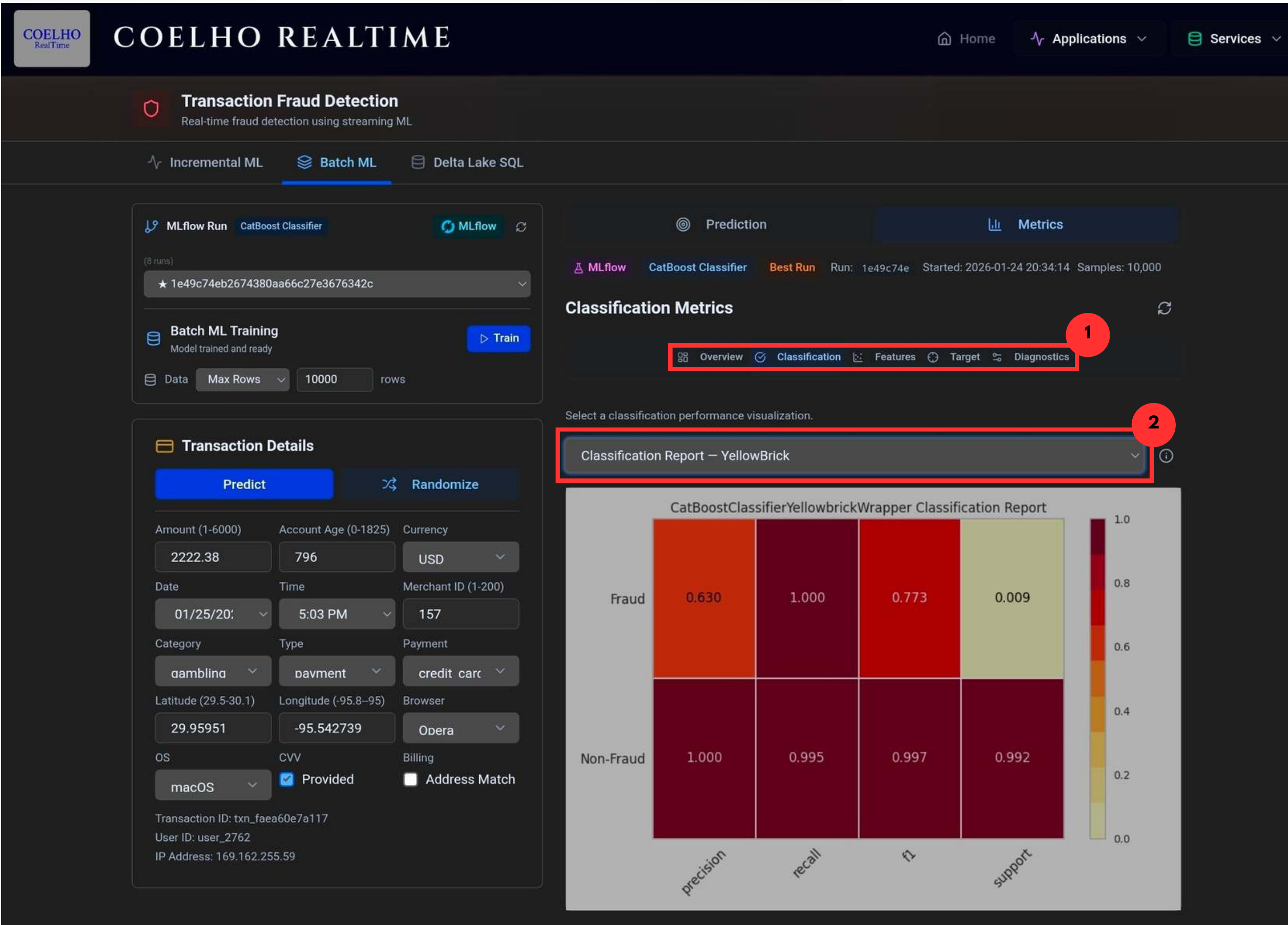
2 Classification metrics tabs

- **Overview:** Displays key MLflow metrics, including primary, ranking, secondary, and calibration scores.
- **Classification:** Provides various visualizations to assess classification model performance.
- **Features:** Offers visual tools for analyzing input features and their properties.
- **Target:** Presents visualizations related to the target variable's distribution and characteristics.
- **Diagnostics:** Contains visualizations for model selection, debugging, and advanced diagnostics.

Metrics (Scikit-Learn library)

- Recall
- Precision
- F1 Score
- F2 ($\beta=2$)
- ROC-AUC
- Average Precision
- Accuracy
- Balanced Accuracy
- Matthews Correlation Coefficient
- Cohen Kappa
- Jaccard
- Geometric Mean
- Log Loss
- Brier Score
- D² Log Loss
- D² Brier

TRANSACTION FRAUD DETECTION - BATCH MACHINE LEARNING



1 Classification Metrics tabs

Overview, Classification, Features, Target, Diagnostics

2 Classification metrics tabs options

- **Classification**
 - Confusion Matrix
 - Classification Report
 - ROC AUC Curve
 - Precision-Recall Curve
 - Class Prediction Error
 - Discrimination Threshold
 - Calibration Curve
 - DET Curve
 - ROC Curve
 - KS Statistic
 - Cumulative Gain
 - Lift Curve
- **Features**
 - Rank 1D
 - Rank 2D
 - PCA Decomposition
 - Manifold
 - Parallel Coordinates
 - RadViz
 - Joint Plot
 - Partial Dependence
 - Decision Boundary
 - PCA 2D Projection
 - PCA Component Variance
 - Feature Importances
- **Target**
 - Class Balance
 - Feature Correlation (Mutual Info)
 - Feature Correlation (Pearson)
 - Balanced Binning Reference
- **Diagnostics**
 - Feature Importances
 - Cross-Validation Scores
 - Validation Curve
 - Learning Curve
 - Recursive Feature Elimination
 - Dropping Curve

TRANSACTION FRAUD DETECTION - DELTA LAKE SQL

The screenshot displays the COELHO REALTIME Transaction Fraud Detection interface. The top navigation bar includes the COELHO REALTIME logo, a home icon, and dropdown menus for Applications and Services. Below the navigation bar, the main header shows 'Transaction Fraud Detection' with a subtitle 'Real-time fraud detection using streaming ML'. The interface is divided into two main sections: 'SQL Editor' and 'Query Results'.

SQL Editor: This section is on the left and contains a text area with the SQL query `SELECT * FROM data LIMIT 100`. A red box labeled '1' highlights the editor area. At the bottom of the editor, there is a blue 'Run' button with a play icon, highlighted by a red box labeled '2'.

Query Results: This section is on the right and displays the results of the SQL query. It includes a search box labeled 'USD' (highlighted by a red box labeled '3'), a row count '10/100', and a latency indicator '1743ms'. Below these, a table shows the query results with columns: timestamp, amount, currency, merchant_id, product_category, and transaction_id. The table contains 10 rows of data. A red box labeled '4' highlights the table content.

timestamp	amount	currency	merchant_id	product_category	transaction_id
2026-01-25T20:36:16.607468+00:00	50.94	USD	merchant_79	luxury_items	pay
2026-01-25T20:36:19.425698+00:00	410.02	USD	merchant_76	luxury_items	def
2026-01-25T20:36:19.956102+00:00	134.35	USD	merchant_126	gambling	wit
2026-01-25T20:36:22.327767+00:00	306.36	USD	merchant_16	travel	def
2026-01-25T20:36:24.058332+00:00	188.02	USD	merchant_36	electronics	wit
2026-01-25T20:36:27.955165+00:00	273.99	USD	merchant_51	services	pay
2026-01-25T20:36:28.646752+00:00	350.2	USD	merchant_77	electronics	tra
2026-01-25T20:36:32.747613+00:00	385.31	USD	merchant_21	clothing	pur
2026-01-25T20:36:37.812369+00:00	436.82	USD	merchant_65	groceries	tra
2026-01-25T20:36:41.739019+00:00	392.15	USD	merchant_123	gambling	p

1 SQL Editor

Interface for writing and editing SQL queries against the TFD Delta Lake.

2 Query run button

Executes the SQL query in the editor against the TFD Delta Lake.

3 Search box, rows and latency

Displays search, allowing user to filter specific values over the entire result, total rows returned, and query execution time.

4 Query results with filter

Presents the results of the SQL query with values filtered on the filter box.

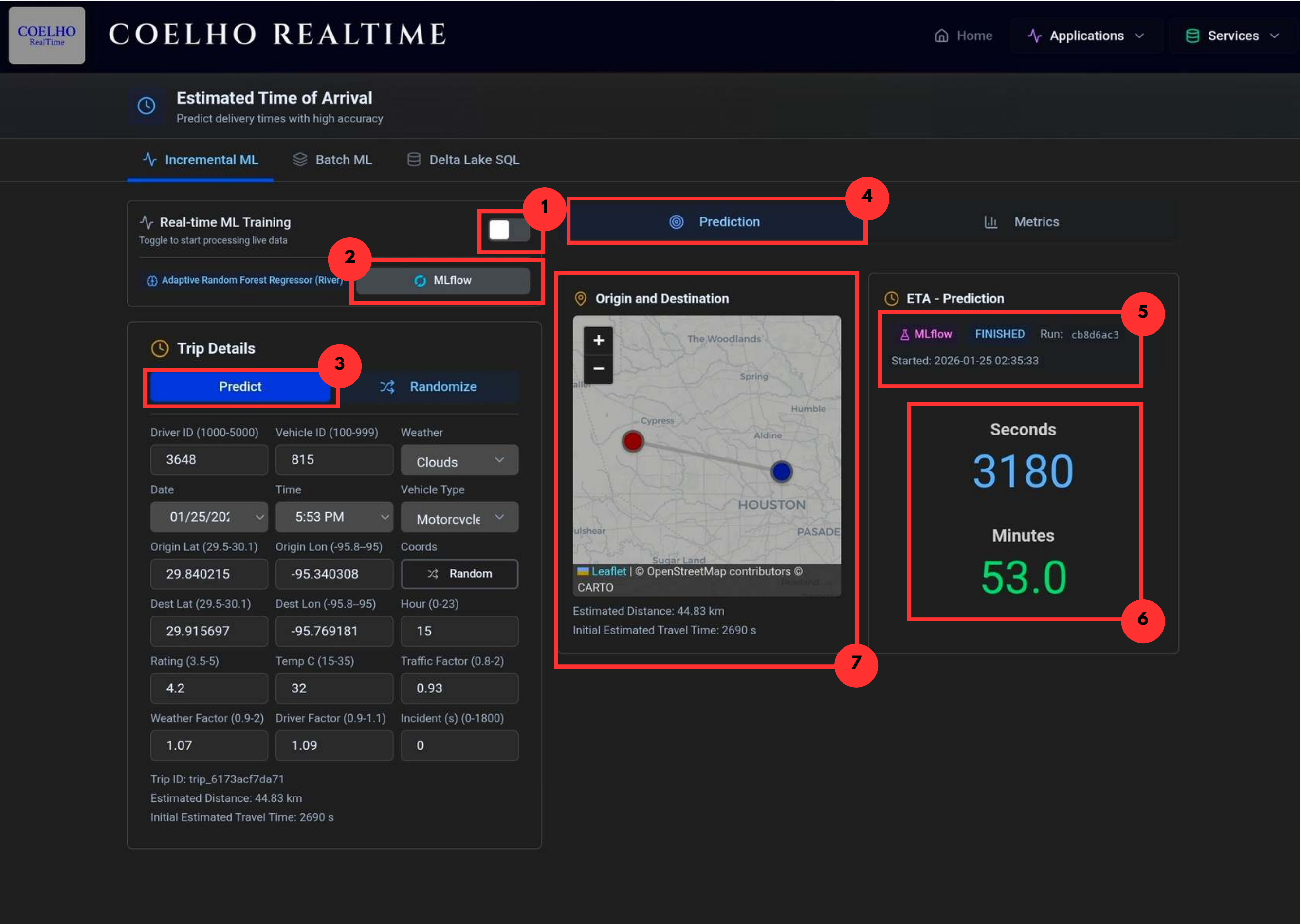
ESTIMATED TIME OF ARRIVAL

Incremental Machine Learning
Batch Machine Learning
Delta Lake SQL

- **Incremental ML model:** Adaptive Random Forest Regressor (River)
- **Batch ML model:** CatBoost Regressor

**COELHO
REALTIME**

ESTIMATED TIME OF ARRIVAL - INCREMENTAL MACHINE LEARNING



1 Real-time ML training toggle

Activates or deactivates the continuous, real-time adaptation of the ETA model, enabling it to learn incrementally from incoming data streams.

2 MLflow model page - experiment run

Provides a summary of a specific incremental learning experiment for ETA on MLflow service page, detailing logged parameters, metrics, and associated artifacts within MLflow.

3 Prediction button

Initiates an immediate ETA prediction using the ETA model's current, incrementally updated state, demonstrating low-latency inference.

4 Prediction tab

Displays real-time estimated time of arrival (ETA) predictions from the incremental machine learning model, offering immediate and continuously updated insights.

5 MLflow experiment run informations

Offers status info (LIVE | FINISHED), MLflow run ID, and the datetime the last real-time ML model training was executed.

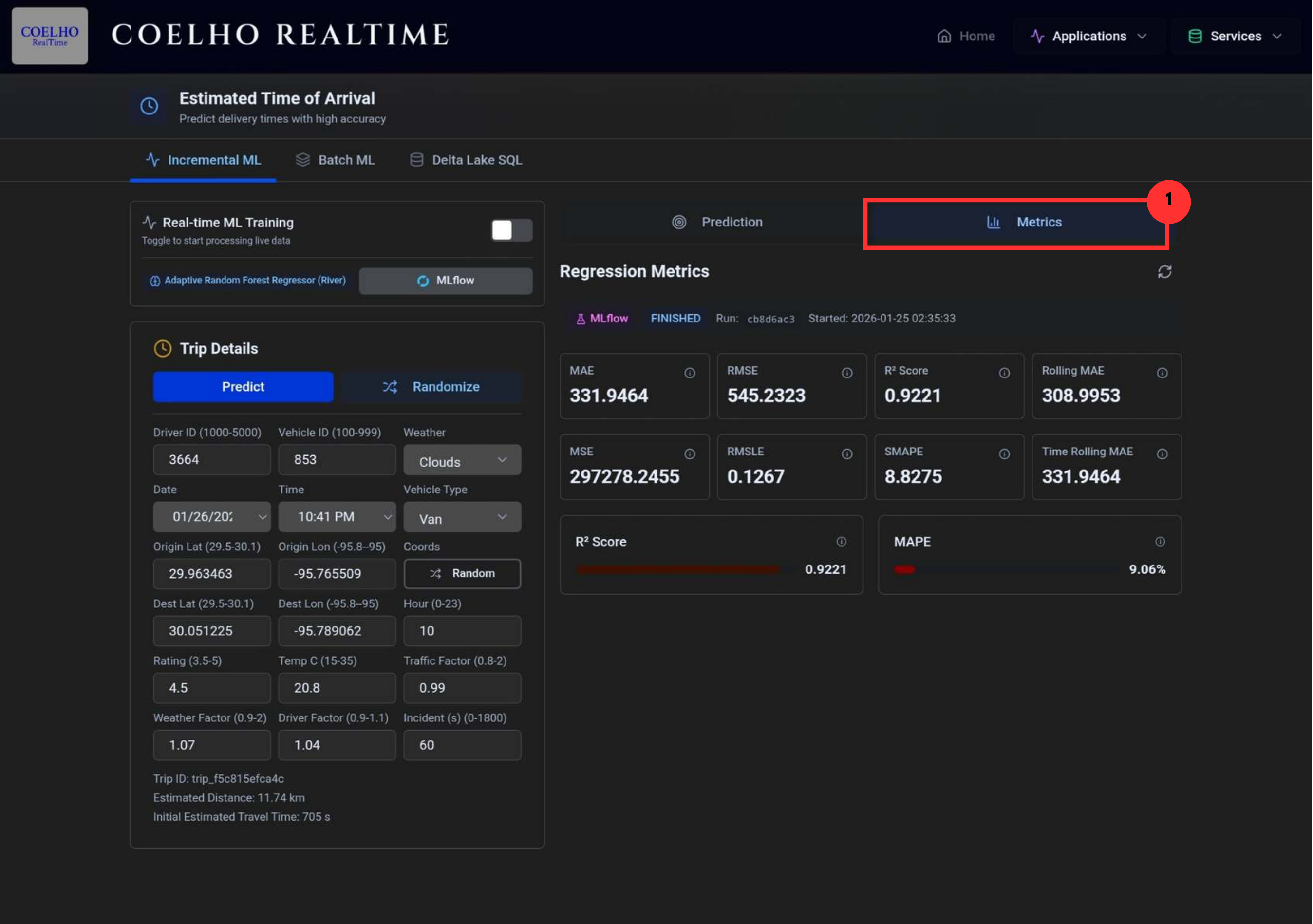
6 Prediction results

Presents the output of the ETA model's real-time predictions.

7 Map with Origin and Destination

Illustrates the real-time or predicted path of an entity, clearly marking its starting point (Origin) and its projected endpoint (Destination) to support dynamic ETA calculations.

ESTIMATED TIME OF ARRIVAL - INCREMENTAL MACHINE LEARNING



1 Metrics tab

Presents the real-time metrics and incremental learning status of the ETA model.

Metrics (River library)

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- Mean Squared Error (MSE)
- R-Squared (R^2)
- Symmetric MAPE (SMAPE)
- Root Mean Squared Log Error (RMSLE)
- Rolling MAE
- Rolling RMSE
- Time Rolling MAE

ESTIMATED TIME OF ARRIVAL - BATCH MACHINE LEARNING

COELHO RealTime

COELHO REALTIME

Home

Applications

Services

Estimated Time of Arrival

Predict delivery times with high accuracy

Incremental ML

Batch ML

Delta Lake SQL

MLflow Run

CatBoost Regressor

MLflow

7a843e6b387a41838261b1dea8269c76

Batch ML Training

Model trained and ready

Data

Max Rows

10000

rows

Train

Trip Details

Predict

Randomize

Driver ID (1000-5000)

3664

Vehicle ID (100-999)

853

Weather

Clouds

Date

01/26/2025

Time

10:41 PM

Vehicle Type

Van

Origin Lat (29.5-30.1)

29.963463

Origin Lon (-95.8-95)

-95.765509

Coords

Random

Dest Lat (29.5-30.1)

30.051225

Dest Lon (-95.8-95)

-95.789062

Hour (0-23)

10

Rating (3.5-5)

4.5

Temp C (15-35)

20.8

Trip ID: trip_f5c815efca4c

Estimated Distance: 11.74 km

Initial ETA: 705 s (11.8 min)

Prediction

Metrics

MLflow

CatBoost Regressor

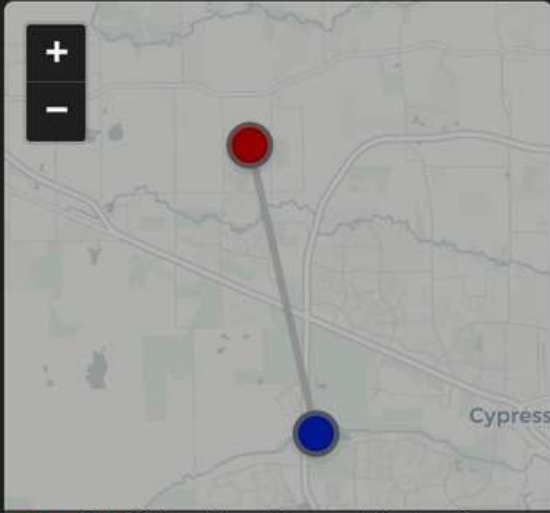
Run: 7a843e6b

Started: 2026-01-23 18:56:58

Origin and Destination

+

-



Estimated Distance: 11.74 km
Initial Estimated Travel Time: 705 s

ETA - Prediction

Seconds

4330

Minutes

72.1

1 Train button

Initiates the batch machine learning training process for the ETA model.

2 MLflow run ID - ML model

Dropdown displaying the MLflow run IDs associated with batch-trained ETA model the user can pick after training new Batch ML models.

3 MLflow run ID page

Navigates to or provides detailed information within MLflow about a particular experiment run.

4 Batch ML Training options

Provides the user the options of training new Batch ML models in real-time per maximum number of rows or per percentage of rows from the original ETA dataset.

5 Prediction button

Triggers a prediction using the currently selected or trained batch ML model.

6 Prediction tab

Displays the interface or section where batch prediction results for the ETA model are presented.

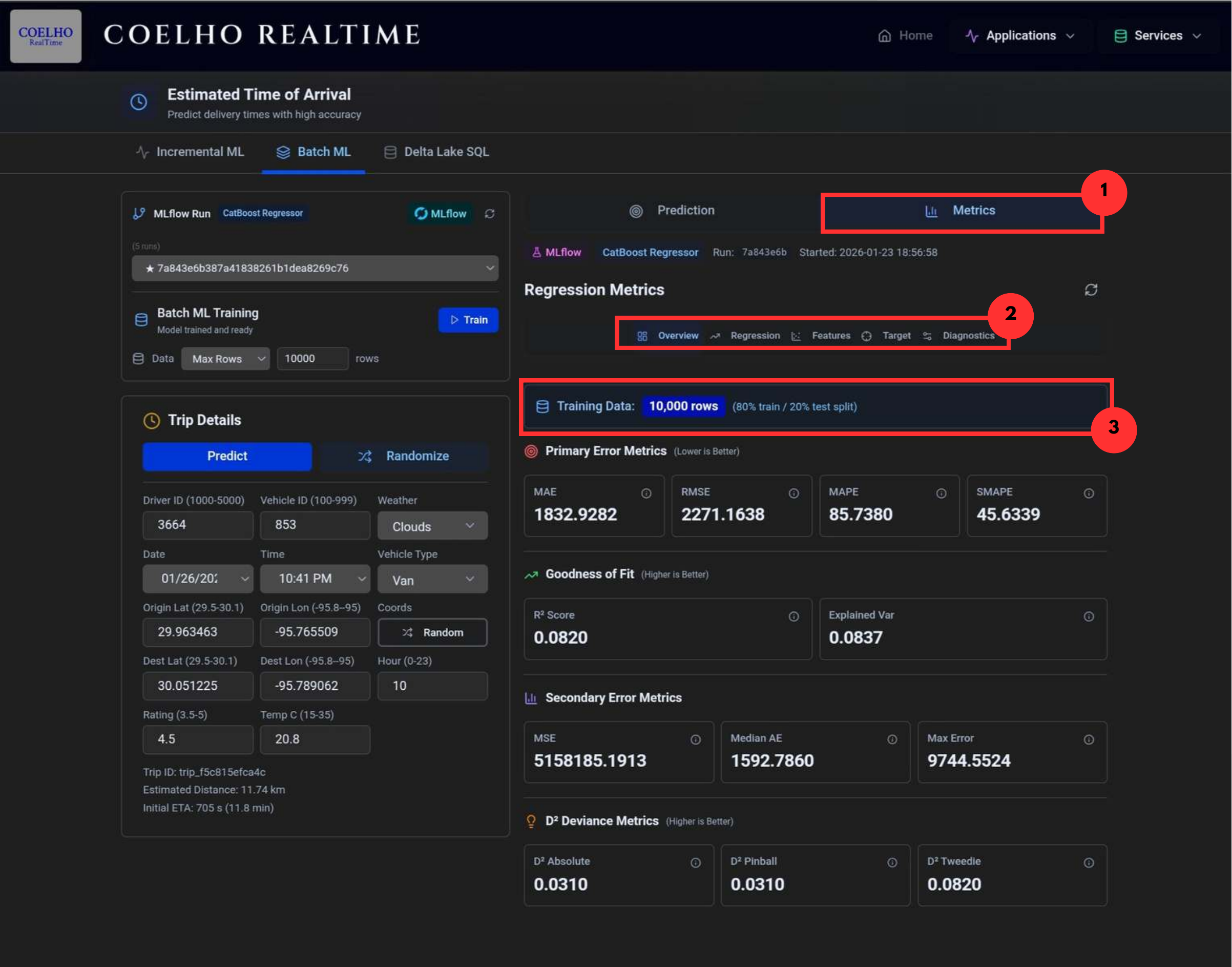
7 MLflow experiment run informations

Offers detailed status information, run ID, and other metadata for an MLflow experiment run.

8 Prediction results

Presents the output of the ETA model's real-time predictions.

ESTIMATED TIME OF ARRIVAL - BATCH MACHINE LEARNING



1 Metrics tab

Presents various metrics of the ETA model. Includes some tabs with a lot of visual metrics from Scikit-Learn, Scikit-Plot and YellowBrick libraries.

2 Regression metrics tabs

- **Overview:** Displays key MLflow metrics, including primary, ranking, secondary, and calibration scores.
- **Regression:** Provides various visualizations to assess regression model performance.
- **Features:** Offers visual tools for analyzing input features and their properties.
- **Target:** Presents visualizations related to the target variable's distribution and characteristics.
- **Diagnostics:** Contains visualizations for model selection, debugging, and advanced diagnostics.

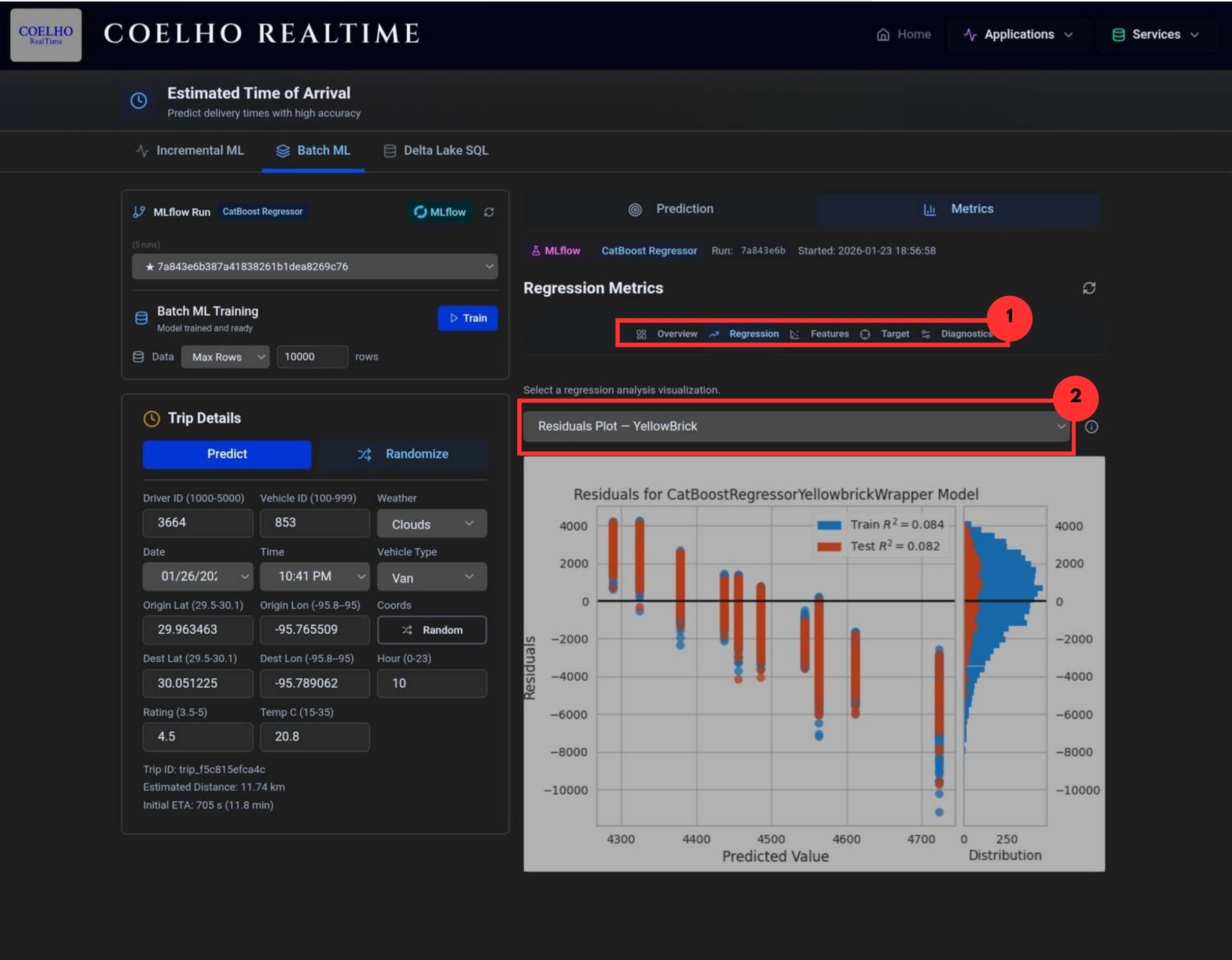
3 Training data info

Displays the total amount of rows used to train the Batch ML model

Metrics (Scikit-Learn library)

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- Mean Absolute Percentage Error (MAPE)
- Symmetric Mean Absolute Percentage Error (SMAPE)
- R2 Score
- Explained Variance Score
- Mean Squared Error (MSE)
- Median Absolute Error (Median AE)
- Max Error
- D2 Absolute Error Score
- D2 Pinball Score
- D2 Tweedie Score

ESTIMATED TIME OF ARRIVAL - BATCH MACHINE LEARNING



1 Regression Metrics tabs

Overview, Regression, Features, Target, Diagnostics

2 Regression metrics tabs options

- **Regression**
 - Residuals Plot
 - Prediction Error
 - Prediction Error (Actual vs Predicted)
 - Prediction Error (Residuals)
- **Features**
 - Rank 1D
 - Rank 2D
 - PCA Decomposition
 - Manifold
 - Joint Plot
 - Partial Dependence
 - PCA 2D Projection
 - PCA Component Variance
- **Target**
 - Feature Correlation (Mutual Info)
 - Feature Correlation (Pearson)
 - Balanced Binning Reference
- **Diagnostics**
 - Feature Importances
 - Cross-Validation Scores
 - Validation Curve
 - Learning Curve
 - Recursive Feature Elimination
 - Dropping Curve

ESTIMATED TIME OF ARRIVAL - DELTA LAKE SQL

COELHO
RealTime

COELHO REALTIME

HomeApplicationsServices

Estimated Time of Arrival
Predict delivery times with high accuracy

Incremental MLBatch MLDelta Lake SQL

SQL EditorDuckDB

SELECT * FROM data LIMIT 100

Run

Query Results

Filter results...100/100825ms

trip_id	driver_id	vehicle_id	timestamp
991be9ad-d1e4-4f3e-bb43-906d72df077c	driver_2319	vehicle_364	2026-01-26T02:55:36.891963+00:00
54b916db-3137-4672-a3d0-b9b59dec922	driver_1838	vehicle_226	2026-01-26T02:55:37.069161+00:00
74010f17-807a-4396-8529-9eb16a12e86e	driver_1662	vehicle_435	2026-01-26T02:55:37.803921+00:00
4d69e773-f204-4b3f-98de-0bc39321e653	driver_1826	vehicle_236	2026-01-26T02:55:38.434321+00:00
9c4e8cff-1e0d-40e4-91dc-22026273e6e7	driver_4833	vehicle_429	2026-01-26T02:55:39.296437+00:00
0d859abf-5d33-4d73-bc77-a7c46357ee47	driver_3155	vehicle_266	2026-01-26T02:55:39.945947+00:00
e5557e0d-eef1-4c4f-9393-be7a00fd140e	driver_2716	vehicle_907	2026-01-26T02:55:40.260966+00:00
87cbd9bd-1a42-4f5c-91c9-f0e8bcefd4b6	driver_1984	vehicle_549	2026-01-26T02:55:41.155048+00:00
63d36cef-78f9-4ae1-927a-f46aedc5df07	driver_3997	vehicle_214	2026-01-26T02:55:41.839967+00:00
69cb9c76-7de5-45f7-bf4a-d91ba42c8082	driver_3075	vehicle_999	2026-01-26T02:55:42.054901+00:00
6837da85-5e86-4b0b-a1a8-f11762ad2189	driver_4215	vehicle_643	2026-01-26T02:55:42.490923+00:00
ad928e6a-dfbd-4aef-9a0b-793b98b07690	driver_1952	vehicle_824	2026-01-26T02:55:43.059951+00:00
771d43a8-5f53-47ff-88db-5f4947c7573d	driver_3630	vehicle_130	2026-01-26T02:55:43.799266+00:00
5bae2167-22cd-45ca-ab0e-4a324ccabe2a	driver_2760	vehicle_290	2026-01-26T02:55:44.747588+00:00
e71ed08c-6ab6-41de-ad82-e7cf6ea0fbc3	driver_2067	vehicle_764	2026-01-26T02:55:45.217021+00:00
8ffb7f38-5b79-473d-9416-f8258be1bdb0	driver_4147	vehicle_131	2026-01-26T02:55:45.325941+00:00
c67a9c4a-9991-4240-82ff-eaf2316ea2eb	driver_1520	vehicle_110	2026-01-26T02:55:45.965612+00:00
732a9f61-1436-4fb6-9455-97ad28788bab	driver_4605	vehicle_952	2026-01-26T02:55:46.920120+00:00
f30471fc-da48-40dc-964f-b1aa3b603b2f	driver_4084	vehicle_373	2026-01-26T02:55:47.142744+00:00
31e11019-137c-4300-a692-33f906946284	driver_2434	vehicle_906	2026-01-26T02:55:47.490762+00:00
89661338-179a-40a8-9778-792b367ecac5	driver_3037	vehicle_956	2026-01-26T02:55:48.040154+00:00
05d52d0b-c73d-48a4-9c2c-543436fc8883	driver_1785	vehicle_796	2026-01-26T02:55:48.271857+00:00
27edb471-8f11-459c-852b-f4c4cbf17bdb	driver_3668	vehicle_151	2026-01-26T02:55:48.822327+00:00
fb95caba-7251-449f-b7fb-5968e294172d	driver_3075	vehicle_605	2026-01-26T02:55:49.540487+00:00
e3509b29-7996-4e46-820a-95fa679645a2	driver_2597	vehicle_164	2026-01-26T02:55:50.079302+00:00
a10d0687-86e3-4362-91ce-5ecbc908a4d5	driver_4986	vehicle_494	2026-01-26T02:55:50.362037+00:00
3d6d58de-c151-4c16-8b26-7b30765d129f	driver_4582	vehicle_797	2026-01-26T02:55:51.288462+00:00
d7f9ca0b-45a4-4ea9-9c75-14f0f84810d0	driver_4516	vehicle_925	2026-01-26T02:55:51.705793+00:00
482cd64b-83b3-45eb-8125-345314e14dc0	driver_1675	vehicle_736	2026-01-26T02:55:52.530936+00:00
11a6f8b1-74fa-4fe1-857c-b8f680bb96ed	driver_1353	vehicle_881	2026-01-26T02:55:53.492551+00:00
f56d583a-20fd-42d4-ab58-f3afbe287a78	driver_2603	vehicle_489	2026-01-26T02:55:53.775921+00:00

1 SQL Editor

Interface for writing and editing SQL queries against the ETA Delta Lake.

2 Query run button

Executes the SQL query in the editor against the ETA Delta Lake.

3 Search box, rows and latency

Displays search, allowing user to filter specific values over the entire result, total rows returned, and query execution time.

4 Query results with filter

Presents the results of the SQL query with values filtered on the filter box.

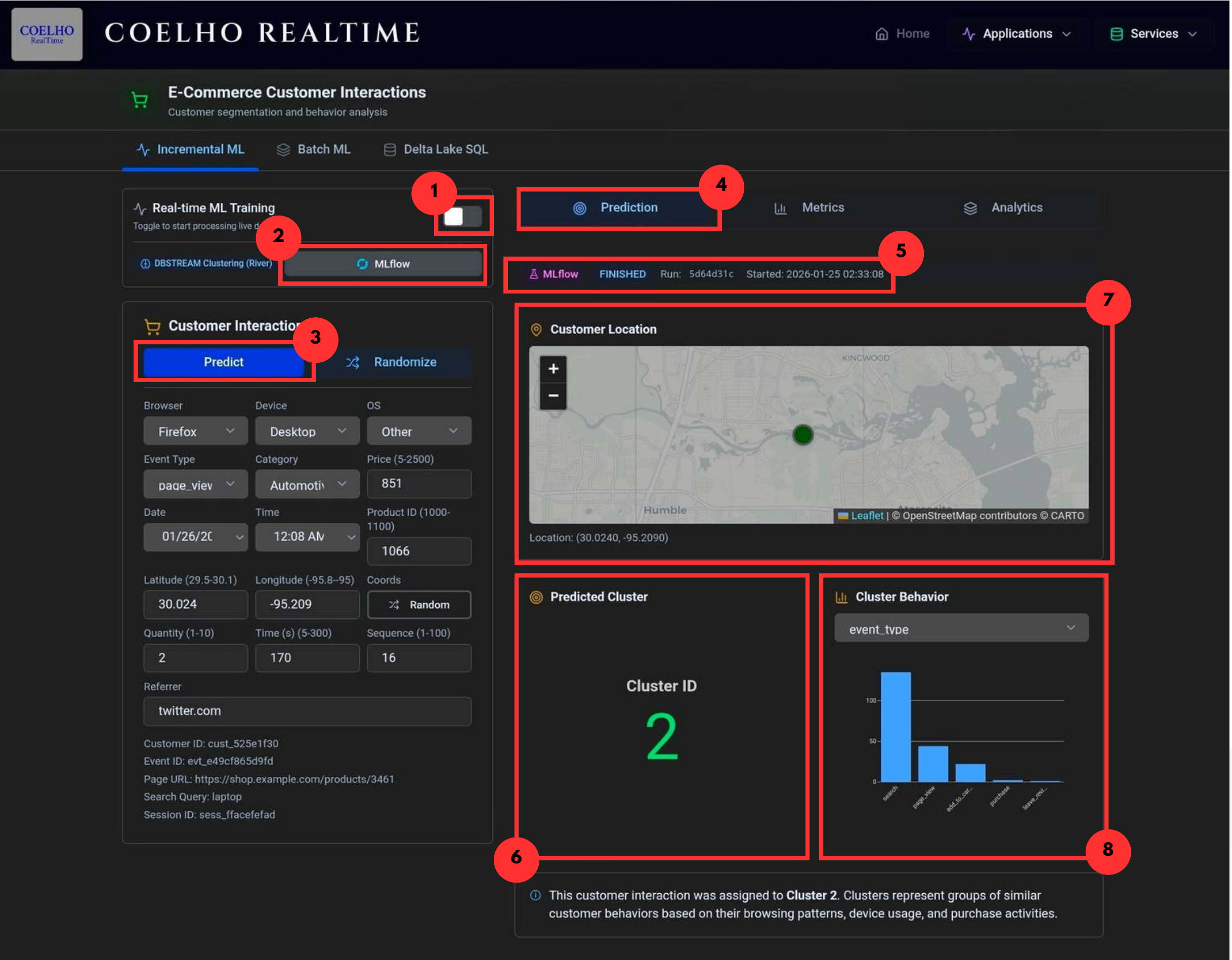
E-COMMERCE CUSTOMER INTERACTIONS

Incremental Machine Learning
Batch Machine Learning
Delta Lake SQL

- Incremental ML model: DBSTREAM (River)
- Batch ML model: KMEANS (Scikit-learn)

**COELHO
REALTIME**

E-COMMERCE CUSTOMER INTERACTIONS - INCREMENTAL MACHINE LEARNING



1 Real-time ML training toggle

Activates or deactivates the continuous, real-time adaptation of the ECCL model, enabling it to learn incrementally from incoming data streams.

2 MLflow model page - experiment run

Provides a summary of a specific incremental learning experiment for ECCL on MLflow service page, detailing logged parameters, metrics, and associated artifacts within MLflow.

3 Prediction button

Initiates an immediate ECCL prediction using the ECCL model's current, incrementally updated state, demonstrating low-latency inference.

4 Prediction tab

Displays real-time ECCL predictions from the incremental machine learning model, offering immediate and continuously updated insights into customer interactions.

5 MLflow experiment run informations

Offers status info (LIVE | FINISHED), MLflow run ID, and the datetime the last real-time ML model training was executed.

6 Prediction results

Presents the output of the ECCL model's real-time predictions.

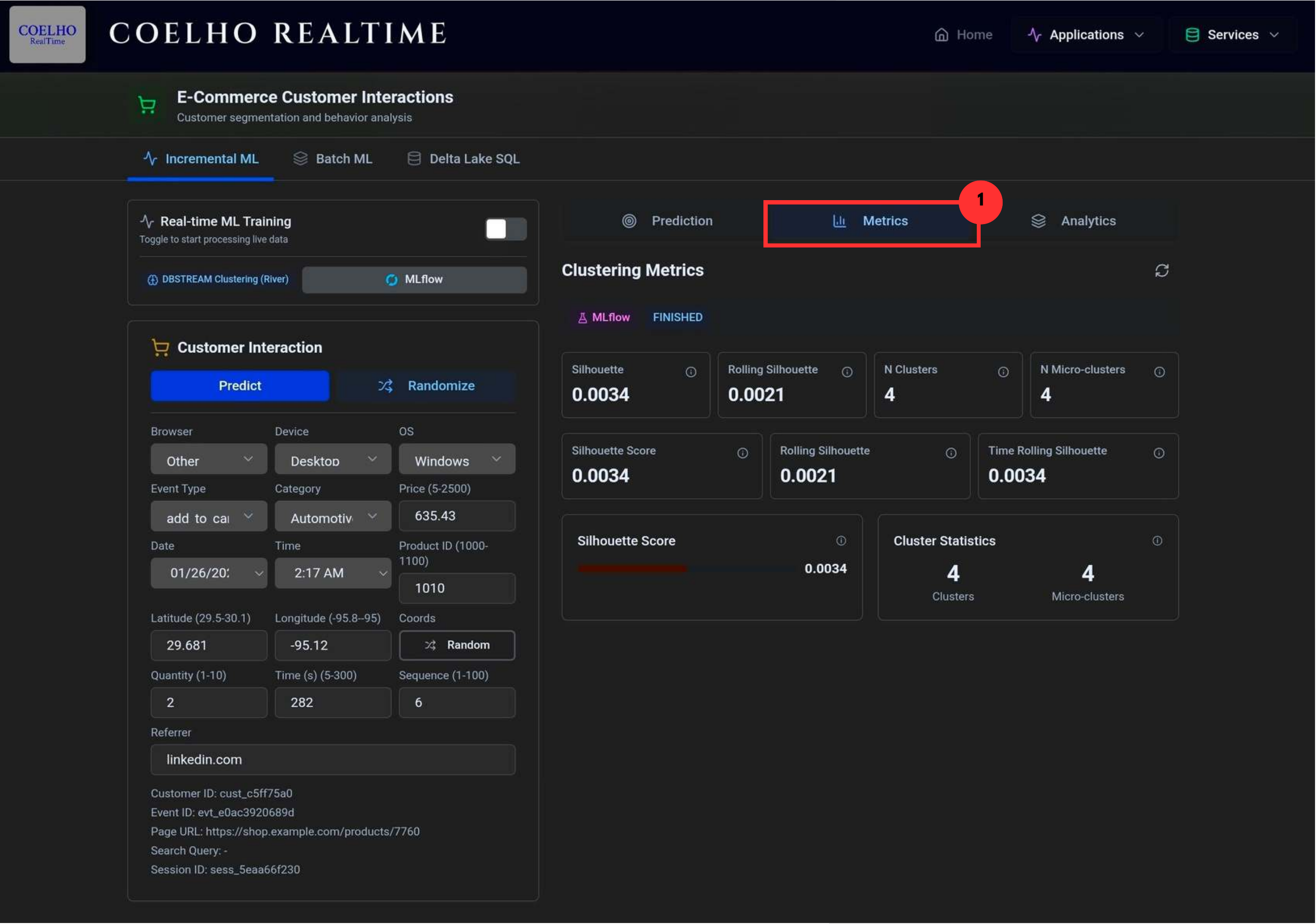
7 Map with customer location

Illustrates customer locations and their interaction journeys on a map, providing spatial insights into e-commerce customer behavior.

8 Cluster behavior analysis

Illustrates the distribution of a selected customer interaction feature (e.g., event type, product category) within the predicted cluster for the sample user, revealing the characteristic behaviors of that customer segment.

E-COMMERCE CUSTOMER INTERACTIONS - INCREMENTAL MACHINE LEARNING



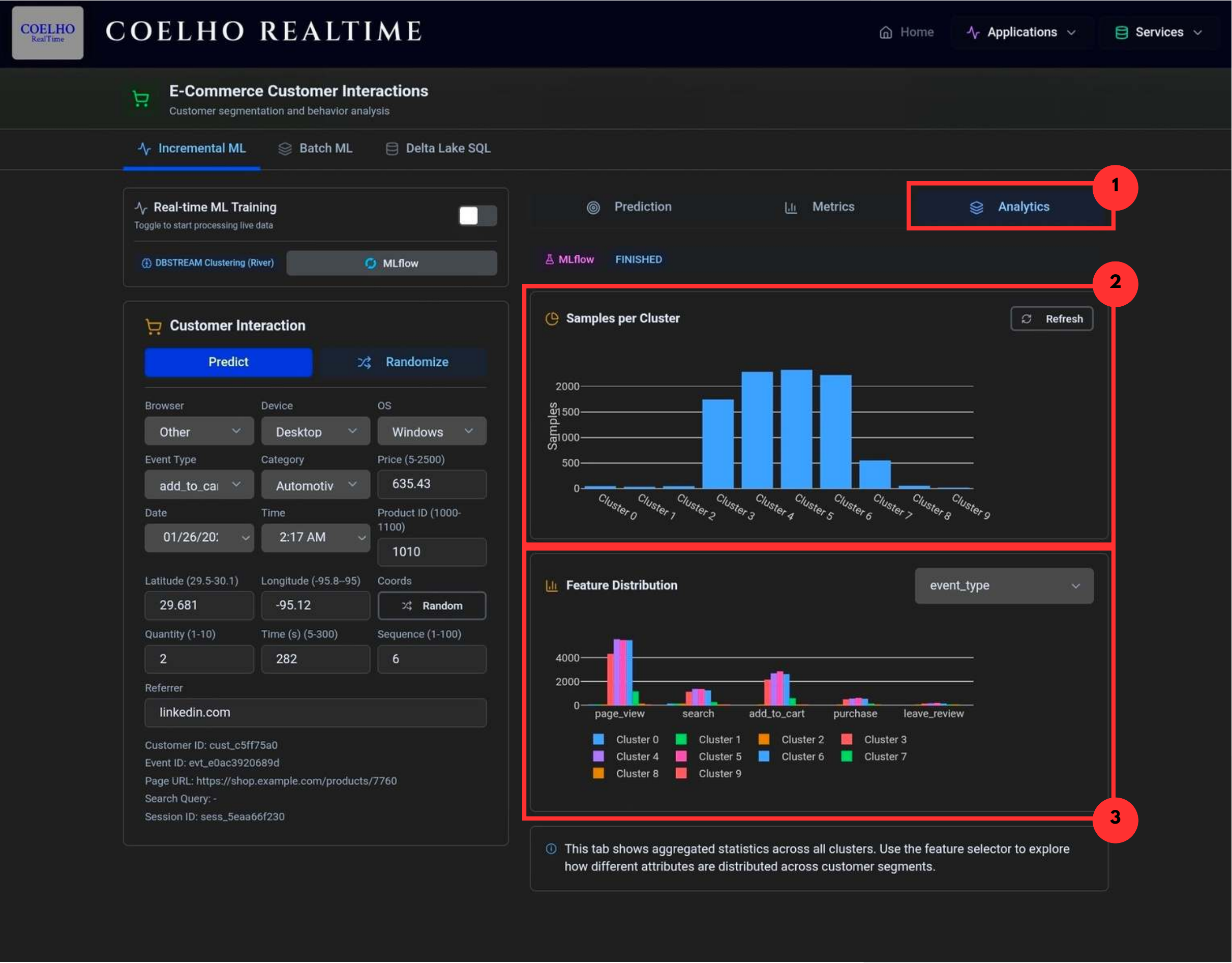
1 Metrics tab

Presents the real-time metrics and incremental learning status of the ECCL model.

Metrics (River library)

- Silhouette
- Silhouette Score
- Rolling Silhouette
- Time Rolling Silhouette
- Number of Clusters
- Number of Micro-clusters

E-COMMERCE CUSTOMER INTERACTIONS - INCREMENTAL MACHINE LEARNING



1 Analytics tab

Provides an in-depth view of aggregated statistics across all identified customer clusters, allowing for comprehensive analysis of customer segments and their evolving characteristics.

2 Samples per cluster

Visualizes the real-time distribution of customer interaction samples across different clusters, showing the relative size and density of each identified customer segment.

3 Feature Distribution

Displays the distribution of values for a selected feature (e.g., event type, product category) across all customer clusters, highlighting the distinguishing attributes and behavioral patterns of each segment.

E-COMMERCE CUSTOMER INTERACTIONS - BATCH MACHINE LEARNING

The screenshot shows the COELHO REALTIME dashboard for E-commerce Customer Interactions. The interface includes a top navigation bar with 'Home', 'Applications', and 'Services' links. The main content area is divided into several sections:

- Batch ML Training:** A section on the left with a 'Train' button (1) and a dropdown for 'Max Rows' (4) set to 10000.
- Customer Interaction:** A form on the left with various input fields (5) for browsing and purchase data, including a 'Predict' button.
- MLflow Run:** A section at the top left showing a list of runs (2) with a dropdown for 'MLflow' (3).
- Prediction:** A tab (6) at the top right that displays a map of 'Customer Location' (7) and a 'Predicted Cluster' (8) with a 'Cluster ID' (1).
- Cluster Behavior:** A chart on the right showing the distribution of customer behaviors.

Numbered callouts (1-8) highlight specific features and actions within the interface.

1 Train button

Initiates the batch machine learning training process for the ECCI model.

2 MLflow run ID - ML model

Dropdown displaying the MLflow run IDs associated with batch-trained ECCI model the user can pick after training new Batch ML models.

3 MLflow run ID page

Navigates to or provides detailed information within MLflow about a particular experiment run.

4 Batch ML Training options

Provides the user the options of training new Batch ML models in real-time per maximum number of rows or per percentage of rows from the original ECCI dataset.

5 Prediction button

Triggers a prediction using the currently selected or trained batch ML model.

6 Prediction tab

Displays the interface or section where batch prediction results for the ECCI model are presented.

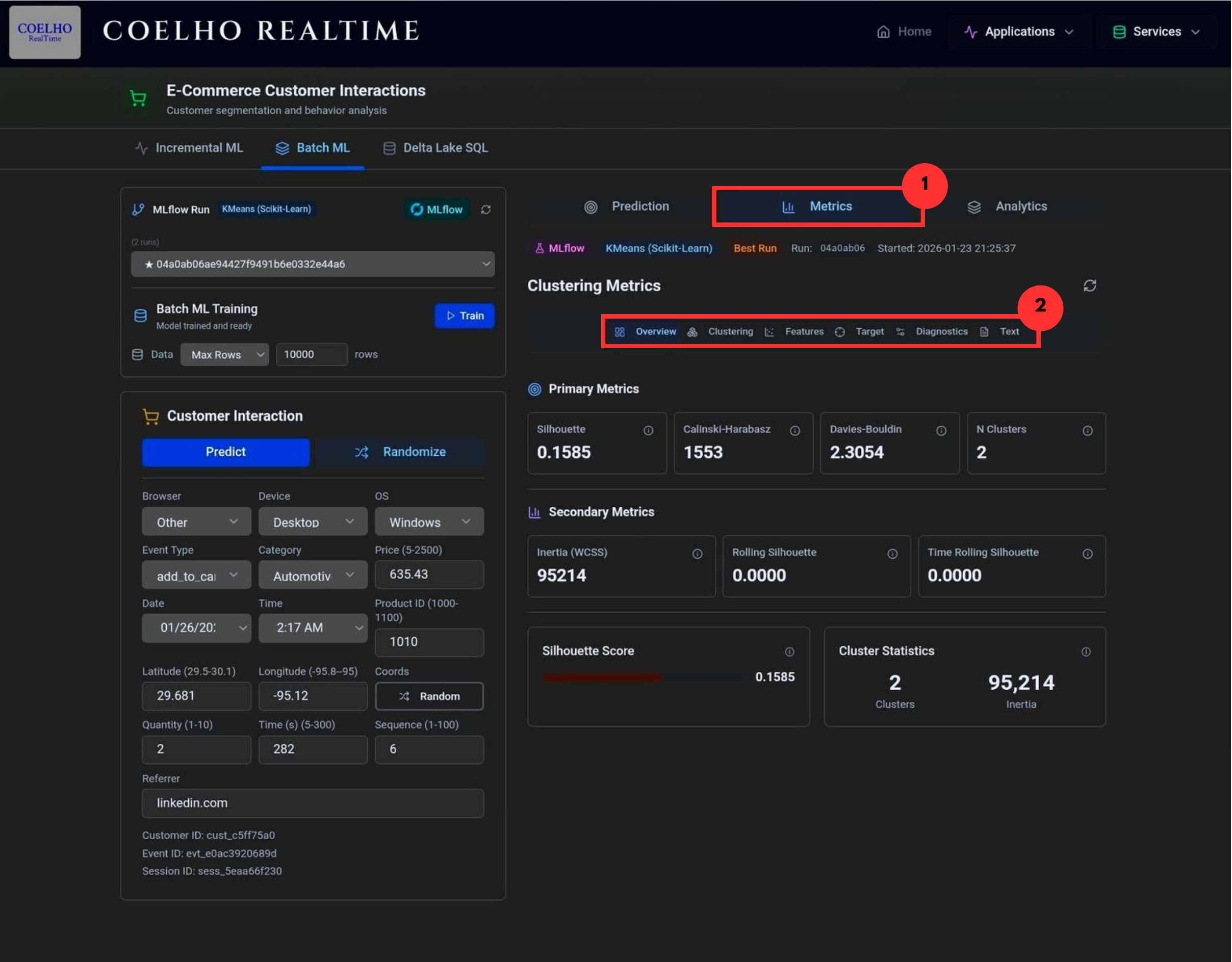
7 MLflow experiment run informations

Offers detailed status information, run ID, and other metadata for an MLflow experiment run.

8 Prediction results

Presents the output of the ECCI model's batch machine learning predictions.

E-COMMERCE CUSTOMER INTERACTIONS - BATCH MACHINE LEARNING



1 Metrics tab

Presents various metrics of the ECCL model. Includes some tabs with a lot of visual metrics from Scikit-Learn, Scikit-Plot and YellowBrick libraries.

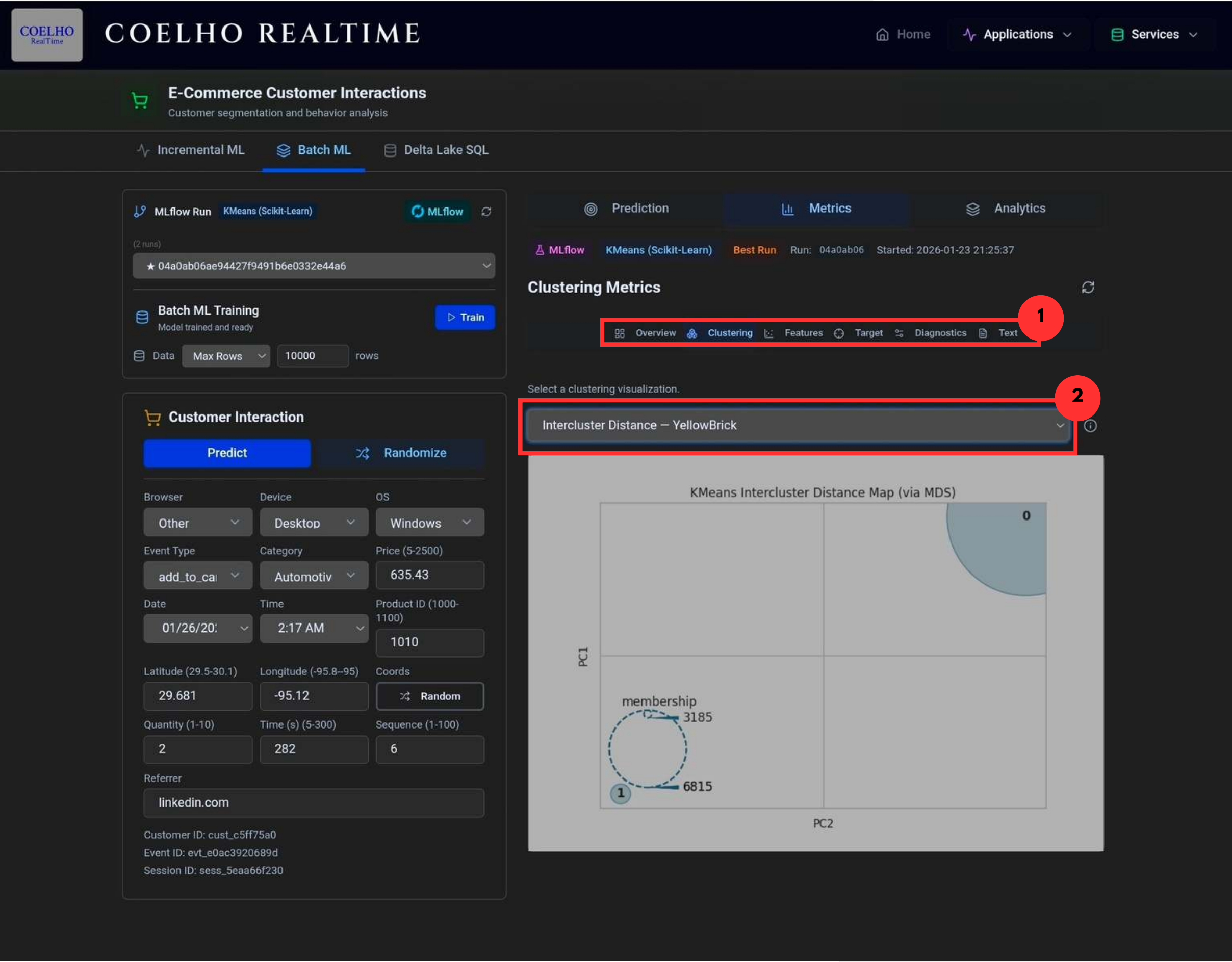
2 Clustering metrics tabs

- **Overview:** Displays key MLflow metrics, including primary, ranking, secondary, and calibration scores.
- **Clustering:** Provides various visualizations to assess clustering model performance.
- **Features:** Offers visual tools for analyzing input features and their properties.
- **Target:** Presents visualizations related to the target variable's distribution and characteristics.
- **Diagnostics:** Contains visualizations for model selection, debugging, and advanced diagnostics.
- **Text:** Visualizations and metrics for Textual Features, specifically for the search_query field, including analyses such as word frequency distributions, search query clustering (t-SNE/UMAP), word dispersion, word correlation, and part-of-speech tagging, all aimed at understanding customer search behavior and intent.

Metrics (Scikit-Learn library)

- Silhouette
- Calinski-Harabasz
- Davies-Bouldin
- N Clusters
- Inertia (WCSS)
- Rolling Silhouette
- Time Rolling Silhouette

E-COMMERCE CUSTOMER INTERACTIONS - BATCH MACHINE LEARNING



1 Clustering Metrics tabs

Overview, Clustering, Features, Target, Diagnostics, Text

2 Clustering metrics tabs options

- Clustering
 - K-Elbow
 - Silhouette
 - Intercluster Distance
 - Elbow Curve
- Features
 - Rank 1D
 - Rank 2D
 - PCA Decomposition
 - Manifold
 - Parallel Coordinates
 - RadViz
 - Joint Plot
 - Partial Dependence
 - Decision Boundary
 - PCA 2D Projection
 - PCA Component Variance
 - Feature Importances
- Target
 - Cluster Distribution
 - Feature Correlation (Mutual Info)
 - Feature Correlation (Pearson)
 - Balanced Binning Reference
- Text
 - Frequency Distribution
 - t-SNE Visualization
 - UMAP Visualization
 - Dispersion Plot
 - Word Correlation
 - POS Tag Distribution
- Diagnostics
 - Feature Importances
 - Cross-Validation Scores
 - Validation Curve
 - Learning Curve
 - Recursive Feature Elimination
 - Dropping Curve

E-COMMERCE CUSTOMER INTERACTIONS - DELTA LAKE SQL

COELHO
RealTime

COELHO REALTIME

HomeApplicationsServices

E-Commerce Customer Interactions
Customer segmentation and behavior analysis

Incremental MLBatch MLDelta Lake SQL

SQL Editor

DuckDB

SELECT product_category, COUNT(*) as count
FROM data GROUP BY product_category ORDER
BY count DESC

Run

Query Results

Filter results...13/133795ms

product_category	count
-	795090
Electronics	416800
Grocery & Gourmet Food	416746
Beauty & Personal Care	416685
Fashion & Apparel	416359
Computers	416001
Home & Garden	415810
Health & Household	415773
Books	415755
Toys & Games	415715
Sports & Outdoors	415696
Pet Supplies	415534
Automotive	415177

Templates

Sample Data

Purchases

1 SQL Editor

Interface for writing and editing SQL queries against the ECCI Delta Lake.

2 Query run button

Executes the SQL query in the editor against the ECCI Delta Lake.

3 Search box, rows and latency

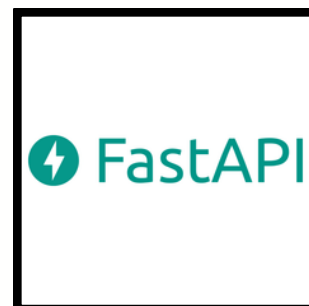
Displays search, allowing user to filter specific values over the entire result, total rows returned, and query execution time.

4 Query results with filter

Presents the results of the SQL query with values filtered on the filter box.

THE MICROSERVICES ENGINE

A closer look at the microservices powering the COELHO RealTime platform — from real-time observability with Prometheus and Grafana, to experiment tracking with MLflow, event streaming with Apache Kafka, and the complete infrastructure that keeps every component running seamlessly on Kubernetes.



FastAPI



Spark



MLflow



MinIO



Prometheus



Grafana

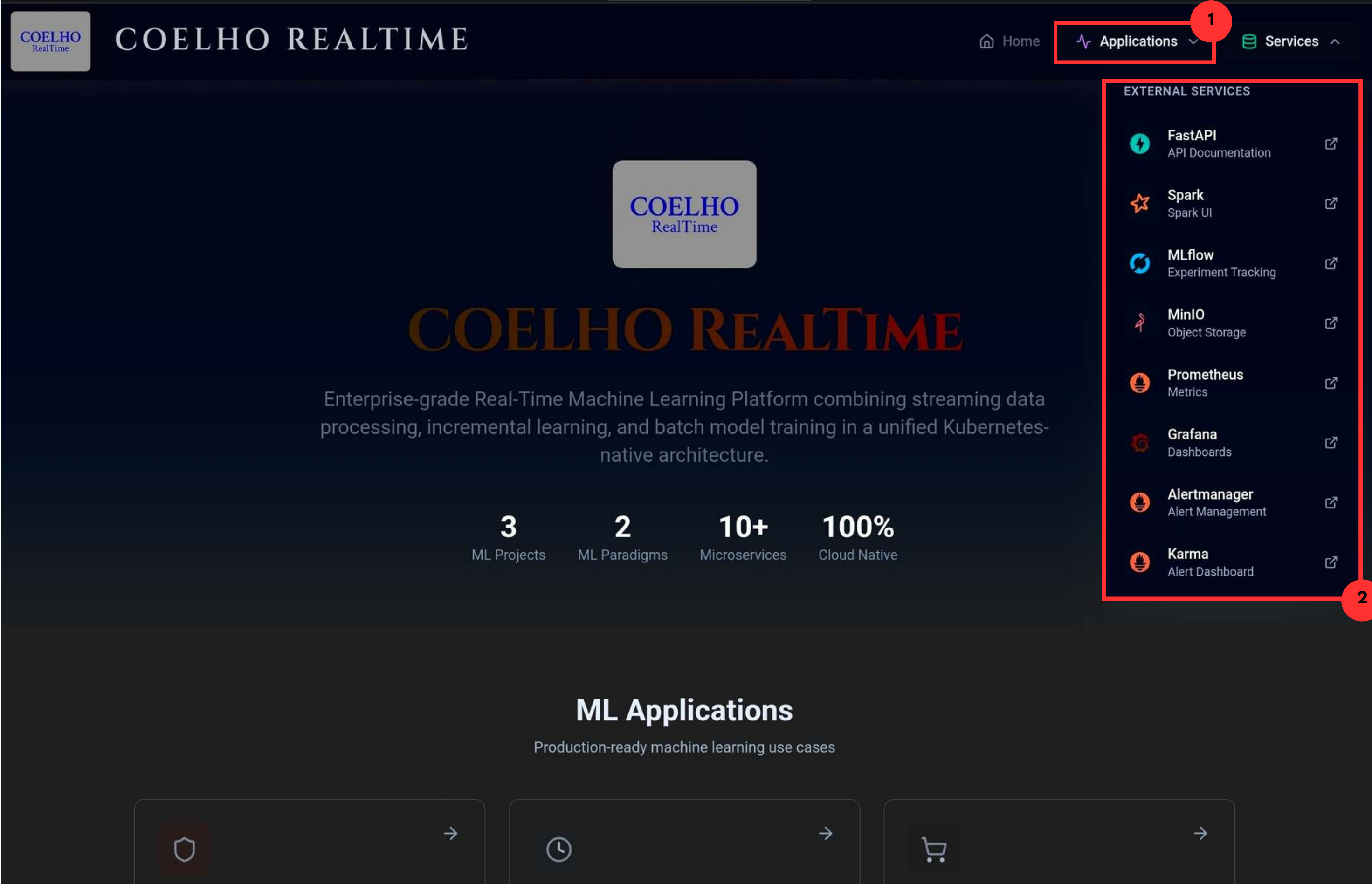


Alertmanager



Karma

MICROSERVICES - HOW TO ACCESS



1 Services dropdown on navbar

The Services dropdown on the navbar provides quick access to all external microservices running on the platform. Clicking it reveals a menu listing 7 services — FastAPI, Spark, MLflow, MinIO, Prometheus, Grafana, and Alertmanager — each linking directly to its web UI in a new browser tab.

2 Microservices options

- **FastAPI** — localhost:8000/docs
- **Spark** — localhost:4040
- **MLflow** — localhost:5001
- **MinIO** — localhost:9001
- **Prometheus** — localhost:9090
- **Grafana** — localhost:3000
- **Alertmanager** — localhost:9094
- **Karma**: localhost:8088

MICROSERVICES - FASTAPI

Unified ML Service - COELHO RealTime

1.0.0OAS 3.1

/openapi.json

Unified API for Incremental ML (River) and Batch ML (Scikit-Learn/CatBoost).

Endpoints

Incremental ML (/api/v1/incremental)

Real-time streaming ML with River library

Kafka consumer integration

Live model predictions during training

Batch ML (/api/v1/batch)

Batch training with CatBoost/Scikit-Learn

YellowBrick visualizations

MLflow model versioning

Delta Lake SQL (/api/v1/sql)

SQL queries against Delta Lake tables

DuckDB and Polars query engines

Table schema inspection

default

GET /metrics Metrics

GET / Root

GET /health Health

Incremental ML (River)

GET /api/v1/incremental/health Health

GET /api/v1/incremental/status Get Status

POST /api/v1/incremental/switch-model Switch Model

GET /api/v1/incremental/training-status/{project_name} Get Training Status

POST /api/v1/incremental/predict Predict

POST /api/v1/incremental/model-available Check Model Available

POST /api/v1/incremental/mlflow-metrics Get Mlflow Metrics

POST /api/v1/incremental/report-metrics Get Report Metrics

1

1 FastAPI – Important REST API Endpoints

Incremental ML (/api/v1/incremental) – 16 endpoints

- [GET] /health – Health check
- [GET] /status – Current training status
- [POST] /switch-model – Start/stop incremental training
- [POST] /predict – Real-time prediction (live Redis or MLflow model)
- [POST] /mlflow-metrics – Experiment metrics with caching
- [POST] /page-init – Combined page initialization
- [POST] /sample – Random dataset sample via DuckDB

Batch ML (/api/v1/batch) – 20 endpoints

- [GET] /health – Health check
- [GET] /status – Training progress with live updates
- [POST] /switch-model – Start/stop batch training
- [POST] /predict – Prediction using MLflow models
- [POST] /mlflow-runs – List all experiment runs
- [POST] /init – Page initialization with all data
- [POST] /visualization/metric/yellowbrick – YellowBrick visualizations
- [POST] /visualization/metric/sklearn – Scikit-Learn visualizations
- [POST] /visualization/metric/scikitplot – Scikit-plot visualizations

Delta Lake SQL (/api/v1/sql) – 3 endpoints

- [POST] /query – Execute SQL against Delta Lake (SELECT only)
- [POST] /schema – Table schema and metadata
- [POST] /total-rows – Total row count

MICROSERVICES - SPARK



Spark Master at spark://coelho-realtime-spark-master-0.coelho-realtime-spark-headless.coelho-realtime.svc.cluster.local:7077

URL: spark://coelho-realtime-spark-master-0.coelho-realtime-spark-headless.coelho-realtime.svc.cluster.local:7077

Workers: 1 Alive, 0 Dead, 0 Decommissioned, 0 Unknown

Cores in use: 8 Total, 6 Used

Memory in use: 3.0 GiB Total, 1536.0 MiB Used

Resources in use:

Applications: 3 Running, 24 Completed

Drivers: 0 Running (0 Waiting), 0 Completed (0 Killed, 0 Failed, 0 Error, 0 Relaunching)

Status: ALIVE (Environment, Log)

Workers (1)

Worker Id	Address	State	Cores	Memory
worker-20260126235202-10.42.0.106-45349	10.42.0.106:45349	ALIVE	8 (6 Used)	3.0 GiB (1536.0 MiB Used)

Running Applications (3)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260127023438-0026 (kill)	KafkaToDelta-ECommerceCustomerInteractions	2	512.0 MiB		2026/01/27 02:34:38	root	RUNNING	8.5 min
app-20260127023438-0025 (kill)	KafkaToDelta-EstimatedTimeOfArrival	2	512.0 MiB		2026/01/27 02:34:38	root	RUNNING	8.5 min
app-20260127023438-0024 (kill)	KafkaToDelta-TransactionFraudDetection	2	512.0 MiB		2026/01/27 02:34:38	root	RUNNING	8.5 min

Completed Applications (24)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260127021321-0021	KafkaToDelta-ECommerceCustomerInteractions	2	512.0 MiB		2026/01/27 02:13:21	root	FINISHED	20 min
app-20260127021322-0022	KafkaToDelta-TransactionFraudDetection	2	512.0 MiB		2026/01/27 02:13:22	root	FINISHED	20 min
app-20260127021322-0023	KafkaToDelta-EstimatedTimeOfArrival	2	512.0 MiB		2026/01/27 02:13:22	root	FINISHED	20 min
app-20260127015433-0020	KafkaToDelta-EstimatedTimeOfArrival	2	512.0 MiB		2026/01/27 01:54:33	root	FINISHED	18 min
app-20260127015433-0018	KafkaToDelta-TransactionFraudDetection	2	512.0 MiB		2026/01/27 01:54:33	root	FINISHED	18 min
app-20260127015433-0019	KafkaToDelta-ECommerceCustomerInteractions	2	512.0 MiB		2026/01/27 01:54:33	root	FINISHED	18 min
app-20260127013721-0016	KafkaToDelta-EstimatedTimeOfArrival	2	512.0 MiB		2026/01/27 01:37:21	root	FINISHED	16 min
app-20260127013721-0015	KafkaToDelta-TransactionFraudDetection	2	512.0 MiB		2026/01/27 01:37:21	root	FINISHED	16 min
app-20260127013722-0017	KafkaToDelta-ECommerceCustomerInteractions	2	512.0 MiB		2026/01/27 01:37:22	root	FINISHED	16 min
app-20260127011310-0013	KafkaToDelta-ECommerceCustomerInteractions	2	512.0 MiB		2026/01/27 01:13:10	root	FINISHED	23 min
app-20260127011310-0012	KafkaToDelta-TransactionFraudDetection	2	512.0 MiB		2026/01/27 01:13:10	root	FINISHED	23 min
app-20260127011310-0014	KafkaToDelta-EstimatedTimeOfArrival	2	512.0 MiB		2026/01/27 01:13:10	root	FINISHED	23 min
app-20260127005700-0009	KafkaToDelta-EstimatedTimeOfArrival	2	512.0 MiB		2026/01/27 00:57:00	root	FINISHED	15 min
app-20260127005701-0011	KafkaToDelta-TransactionFraudDetection	2	512.0 MiB		2026/01/27 00:57:01	root	FINISHED	15 min
app-20260127005701-0010	KafkaToDelta-ECommerceCustomerInteractions	2	512.0 MiB		2026/01/27 00:57:01	root	FINISHED	15 min
app-20260127003704-0007	KafkaToDelta-EstimatedTimeOfArrival	2	512.0 MiB		2026/01/27 00:37:04	root	FINISHED	19 min
app-20260127003704-0006	KafkaToDelta-ECommerceCustomerInteractions	2	512.0 MiB		2026/01/27 00:37:04	root	FINISHED	19 min
app-20260127003704-0008	KafkaToDelta-TransactionFraudDetection	2	512.0 MiB		2026/01/27 00:37:04	root	FINISHED	19 min
app-20260127001818-0004	KafkaToDelta-ECommerceCustomerInteractions	2	512.0 MiB		2026/01/27 00:18:18	root	FINISHED	18 min
app-20260127001817-0003	KafkaToDelta-TransactionFraudDetection	2	512.0 MiB		2026/01/27 00:18:17	root	FINISHED	18 min
app-20260127001818-0005	KafkaToDelta-EstimatedTimeOfArrival	2	512.0 MiB		2026/01/27 00:18:18	root	FINISHED	18 min
app-20260126235417-0002	KafkaToDelta-EstimatedTimeOfArrival	2	512.0 MiB		2026/01/26 23:54:17	root	FINISHED	23 min

MICROSERVICES - MLFLOW

mlflow

3.7.0

+ New

Home

Experiments

Models

Prompts

Welcome to MLflow

Get started

Log traces

Trace LLM applications for debugging and monitoring.

Run evaluation

Iterate on quality with offline evaluations and comparisons.

Train models

Track experiments, parameters, and metrics throughout training.

Register prompts

Manage prompt updates and collaborate across teams.

Experiments

	Name	Time created	Last modified	Description	Tags
☐	Default	Invalid Date	Invalid Date	-	
☐	Transaction Fraud Detection	01/19/2026, 10:31:18 AM	01/19/2026, 06:41:32 PM	-	
☐	E-Commerce Customer Interactions	01/19/2026, 06:38:31 PM	01/19/2026, 06:38:31 PM	-	
☐	Estimated Time of Arrival	01/19/2026, 06:38:30 PM	01/19/2026, 06:38:30 PM	-	

Discover new features

MLflow MCP server

Connect your coding assistants and AI applications to MLflow and automatically analyze your experiments and traces.

Optimize prompts

Access the state-of-the-art prompt optimization algorithms such as MIPROv2, GEPA, through MLflow Prompt Registry.

Agents-as-a-judge

Leverage agents as a judge to perform deep trace analysis and improve your evaluation accuracy.

Dataset tracking

Track dataset lineage and versions and effectively drive the quality improvement loop.

1

MLflow experiments

Each ML project maps to a dedicated MLflow experiment: Transaction Fraud Detection (binary classification), Estimated Time of Arrival (regression), and E-Commerce Customer Interactions (clustering).

Both Incremental (River) and Batch (Scikit-Learn/CatBoost) training runs are logged under their respective experiment, tracking parameters, metrics, and artifacts across model versions.

MICROSERVICES - MLFLOW

Transaction Fraud Detection > Runs >

CatBoostClassifier

Overview

Model metrics

System metrics

Traces

Artifacts

Description

No description

Metrics (17)

Search metrics

Metric	Value	Models
preprocessing_time_seconds	94.86727929115295	model
recall_score	0.9166666666666666	model
precision_score	0.049107142857142856	model
f1_score	0.09322033898305085	model
fbeta_score	0.20220588235294118	model
accuracy_score	0.786	model
balanced_accuracy_score	0.8505398110661269	model
matthews_corrcoef	0.18309714371605	model
cohen_kappa_score	0.07208269737754969	model
jaccard_score	0.04888888888888889	model
roc_auc_score	0.9031713900134952	model
average_precision_score	0.48358252164502163	model
log_loss	0.6599173599219417	model
brier_score_loss	0.23340218826303138	model

Parameters (60)

Search parameters

About this run

Created at

01/24/2026, 11:52:12 PM

Created by

root

Experiment ID

2

Status

Finished

Run ID

9febb65a09f84d358c00ced498b56fb5

Duration

14.5s

Source

tfd.py

Registered prompts

Datasets

None

Tags

training_mode: batch preprocessing: DuckDB SQL

Registered models

None

1

MLflow experiment run metrics

Metrics are logged in real time during training and persist after completion. For classification (TFD): F-beta, accuracy, precision, recall, F1 score. For regression (ETA): RMSE, MAE, R². For clustering (ECCI): Silhouette score, inertia, number of clusters.

These metrics enable comparison across runs and determine which model is selected for predictions.

2

MLflow experiment run info

MICROSERVICES - MLFLOW

mlflow 3.7.0

Transaction Fraud Detection > Runs >

CatBoostClassifier

Overview Model metrics System metrics Traces Artifacts

training_data

visualizations

classifier

ClassPredictionError.png

ClassificationReport.png

ConfusionMatrix.png

DiscriminationThreshold.png

PrecisionRecallCurve.png

ROCAUC.png

features

model_selection

scikitplot

sklearn

target

CatBoostClassifier.pkl

sklearn_encoders.pkl

visualizations/classifier/ConfusionMatrix.png 23.85KB

Path: s3://mlflow-artifacts/2/1e49c74eb2674380aa66c27e3676342c/artifacts/visualizations/classifier/ConfusionMatrix.png

CatBoostClassifierYellowbrickWrapper Confusion Matrix

True Class	Non-Fraud	99%	1%
	Fraud	0%	100%
	Non-Fraud	Predicted Class	
		Fraud	

1

MLflow experiment run artifacts

Each run stores the trained model (pickle), feature encoders, training data (Parquet), and visualizations generated by YellowBrick, Scikit-Learn, and Scikit-plot (e.g., confusion matrix, ROC curve, residuals plot, silhouette analysis).

Artifacts are saved to MinIO (S3-compatible storage) and loaded on demand for predictions, model comparison, and visualization caching.

MICROSERVICES - MINIO

MINIO

OBJECT STORE

AGPL LICENSE

User

Object Browser

Access Keys

Documentation

Administrator

Buckets

Policies

Identity

Monitoring

Events

Configuration

License

Object Browser

Filter Buckets

Name	Objects	Size	Access
lakehouse	336,564	12.6 GiB	R/W
mlflow-artifacts	748	145.5 MiB	R/W

1

MinIO buckets

MinIO serves as the unified storage backend for the platform.

The mlflow-artifacts bucket stores all MLflow experiment data – trained models, encoders, training datasets, and visualizations.

The lakehouse bucket holds the Delta Lake tables consumed by Apache Spark and DuckDB, organized by project (TFD, ETA, ECCI).

Both buckets are accessed via the S3 API, making the setup portable to any S3-compatible cloud provider.

MICROSERVICES - PROMETHEUS

Prometheus

Query

Alerts

Status

up{namespace="coelho-realtime"}

Execute

Table

Graph

Explain

Evaluation time

Load time: 105ms Result series: 20

up(instance="coelho-realtime-minio.coelho-realtime", job="coelho-realtime-minio", namespace="coelho-realtime")	1
up(container="grafana", endpoint="http-web", instance="10.42.1.245:3000", job="coelho-realtime-grafana", namespace="coelho-realtime", pod="coelho-realtime-grafana-5f478675b8-rftjt", service="coelho-realtime-grafana")	1
up(container="coelho-realtime-kafka-producers-container", endpoint="metrics", instance="10.42.1.243:8000", job="coelho-realtime-kafka-producers", namespace="coelho-realtime", pod="coelho-realtime-kafka-producers-577f7b7c86-sss8m", service="coelho-realtime-kafka-producers")	1
up(container="spark-streaming", endpoint="spark-ui-eta", instance="10.42.1.238:4041", job="coelho-realtime-spark-streaming", namespace="coelho-realtime", pod="coelho-realtime-spark-streaming-6b75cd7f7-vdfwt", service="coelho-realtime-spark-streaming")	1
up(container="node-exporter", endpoint="http-metrics", instance="172.23.0.5:9100", job="node-exporter", namespace="coelho-realtime", pod="coelho-realtime-prometheus-node-exporter-2vblp", service="coelho-realtime-prometheus-node-exporter")	1
up(container="spark-streaming", endpoint="spark-ui-eccl", instance="10.42.1.238:4042", job="coelho-realtime-spark-streaming", namespace="coelho-realtime", pod="coelho-realtime-spark-streaming-6b75cd7f7-vdfwt", service="coelho-realtime-spark-streaming")	1
up(container="mlflow", endpoint="http", instance="10.42.0.105:5000", job="coelho-realtime-mlflow", namespace="coelho-realtime", pod="coelho-realtime-mlflow-65b6d999f5-pm86f", service="coelho-realtime-mlflow")	1
up(container="spark-master", endpoint="http", instance="10.42.0.107:8080", job="coelho-realtime-spark-master-svc", namespace="coelho-realtime", pod="coelho-realtime-spark-master-0", service="coelho-realtime-spark-master-svc")	1
up(container="spark-streaming", endpoint="spark-ui-tfd", instance="10.42.1.238:4040", job="coelho-realtime-spark-streaming", namespace="coelho-realtime", pod="coelho-realtime-spark-streaming-6b75cd7f7-vdfwt", service="coelho-realtime-spark-streaming")	1
up(container="node-exporter", endpoint="http-metrics", instance="172.23.0.2:9100", job="node-exporter", namespace="coelho-realtime", pod="coelho-realtime-prometheus-node-exporter-gz4gg", service="coelho-realtime-prometheus-node-exporter")	1
up(endpoint="http", instance="10.42.1.242:8000", job="coelho-realtime-fastapi", namespace="coelho-realtime", pod="coelho-realtime-fastapi-6bbb9b9b88-9z7kt", service="coelho-realtime-fastapi")	1
up(container="metrics", endpoint="http-metrics", instance="10.42.1.241:9121", job="coelho-realtime-redis-metrics", namespace="coelho-realtime", pod="coelho-realtime-redis-master-0", service="coelho-realtime-redis-metrics")	1
up(container="kube-prometheus-stack", endpoint="https", instance="10.42.1.240:10250", job="coelho-realtime-kube-prome-operator", namespace="coelho-realtime", pod="coelho-realtime-kube-prome-operator-5548f495c8-djnj5", service="coelho-realtime-kube-prome-operator")	1
up(container="alertmanager", endpoint="http-web", instance="10.42.1.246:9093", job="coelho-realtime-kube-prome-alertmanager", namespace="coelho-realtime", pod="alertmanager-coelho-realtime-kube-prome-alertmanager-0", service="coelho-realtime-kube-prome-alertmanager")	1
up(container="metrics", endpoint="http-metrics", instance="10.42.0.112:9187", job="coelho-realtime-postgresql-metrics", namespace="coelho-realtime", pod="coelho-realtime-postgresql-0", service="coelho-realtime-postgresql-metrics")	1
up(container="minio", endpoint="http", instance="10.42.0.111:9000", job="coelho-realtime-minio", namespace="coelho-realtime", pod="coelho-realtime-minio-7cf4ff575d-gppfv", service="coelho-realtime-minio")	1
up(container="config-reloader", endpoint="reloader-web", instance="10.42.1.246:8080", job="coelho-realtime-kube-prome-alertmanager", namespace="coelho-realtime", pod="alertmanager-coelho-realtime-kube-prome-alertmanager-0", service="coelho-realtime-kube-prome-alertmanager")	1
up(container="kube-state-metrics", endpoint="http", instance="10.42.1.244:8080", job="kube-state-metrics", namespace="coelho-realtime", pod="coelho-realtime-kube-state-metrics-78d45896c6-7jbzk", service="coelho-realtime-kube-state-metrics")	1
up(container="config-reloader", endpoint="reloader-web", instance="10.42.1.247:8080", job="coelho-realtime-kube-prome-prometheus", namespace="coelho-realtime", pod="prometheus-coelho-realtime-kube-prome-prometheus-0", service="coelho-realtime-kube-prome-prometheus")	1
up(container="prometheus", endpoint="http-web", instance="10.42.1.247:9090", job="coelho-realtime-kube-prome-prometheus", namespace="coelho-realtime", pod="prometheus-coelho-realtime-kube-prome-prometheus-0", service="coelho-realtime-kube-prome-prometheus")	1

+ Add query

1

PromQL query

Prometheus Query Language expression executed against the time-series database.

The up metric is a built-in indicator that returns 1 if a target is reachable and 0 if it is down, serving as the foundation for all service health alerts in the platform.

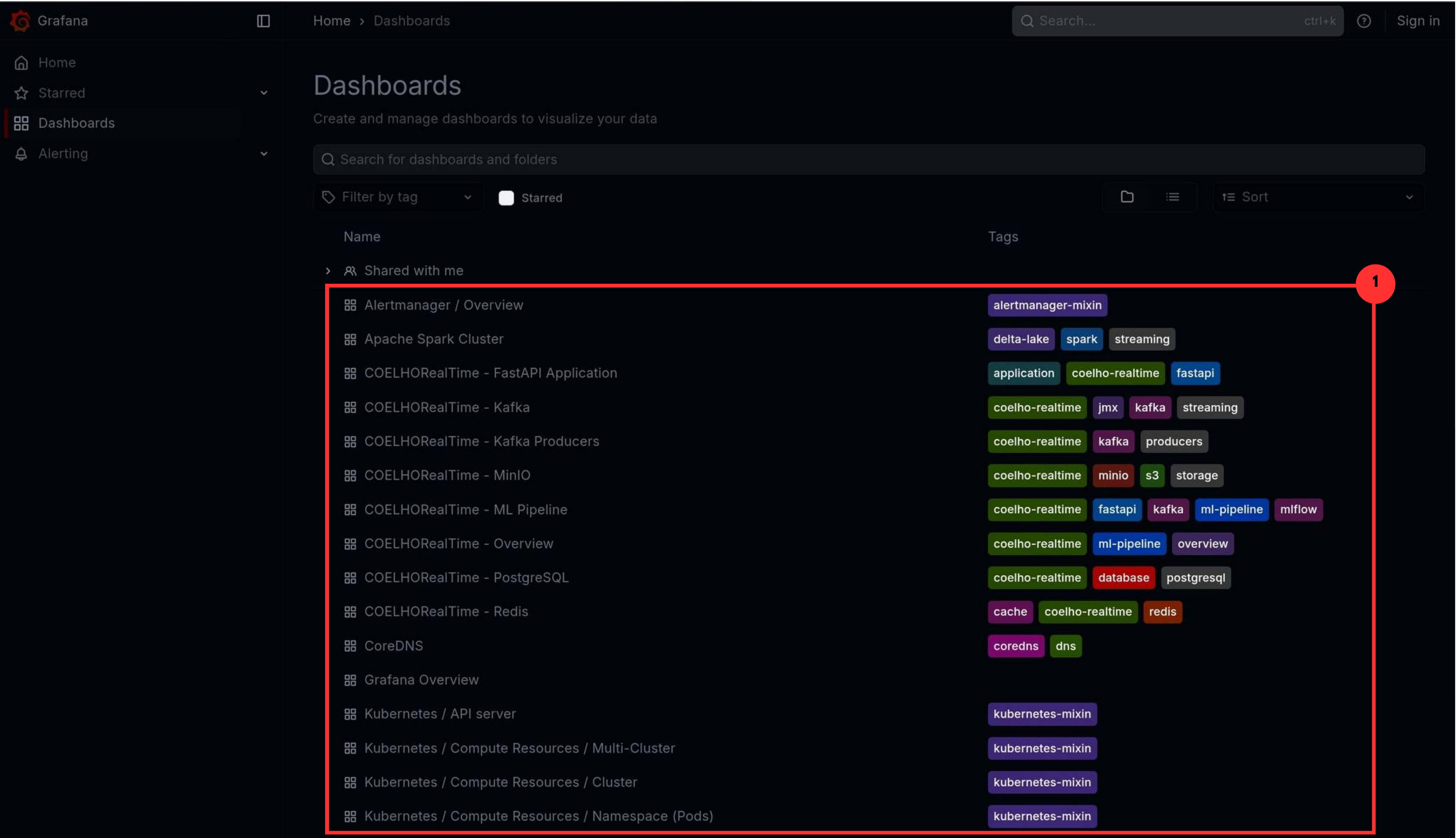
2

PromQL query results

The query returns 20 monitored targets across the platform: FastAPI, Kafka Producers, Spark (Master + 3 Streaming jobs), MLflow, MinIO, Redis, PostgreSQL, Grafana, Alertmanager, Prometheus, Kube State Metrics, and Node Exporters — all reporting up = 1 (healthy).

These results feed the alerting rules that trigger notifications when any service goes down.

MICROSERVICES - GRAFANA

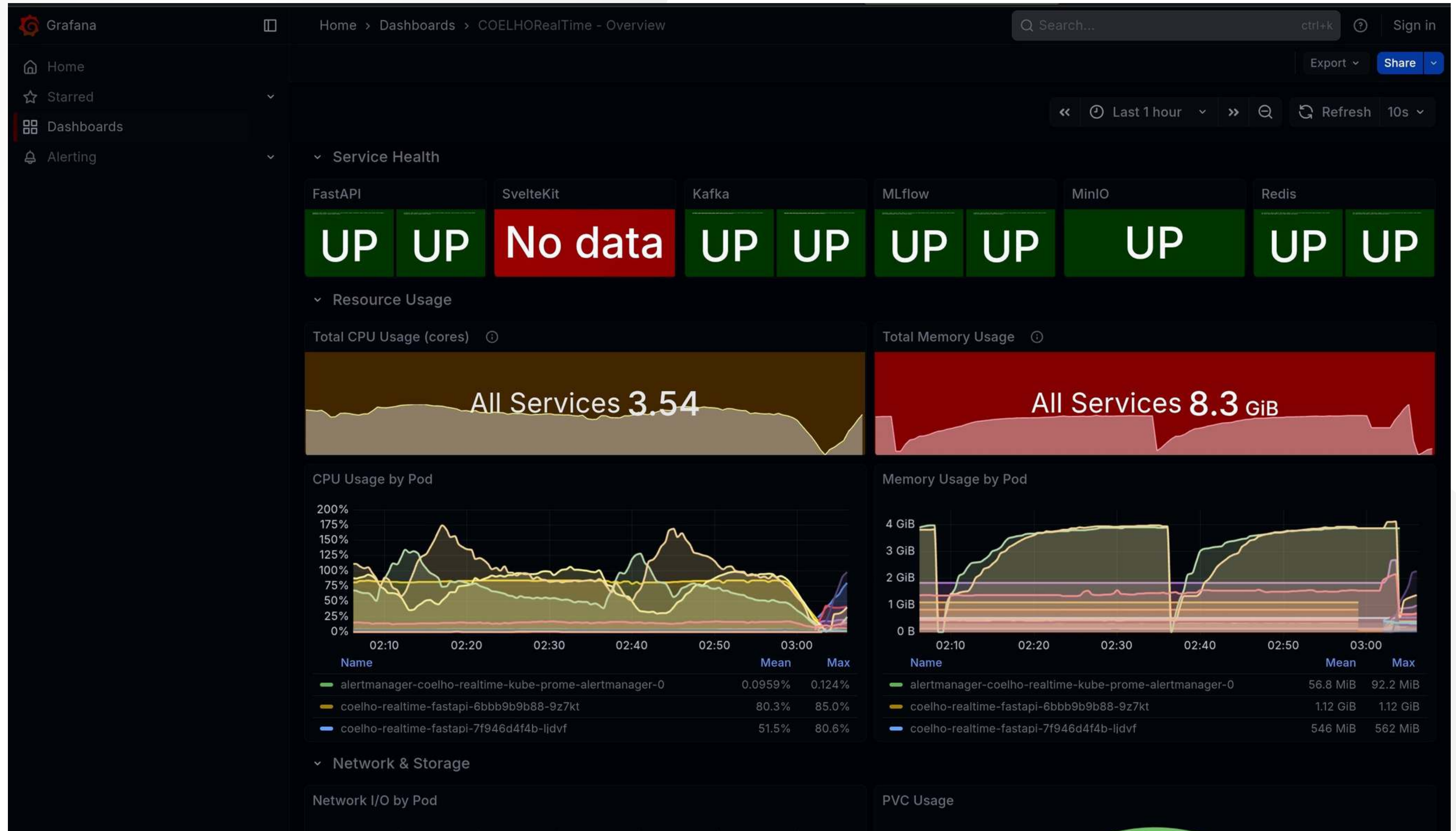


1

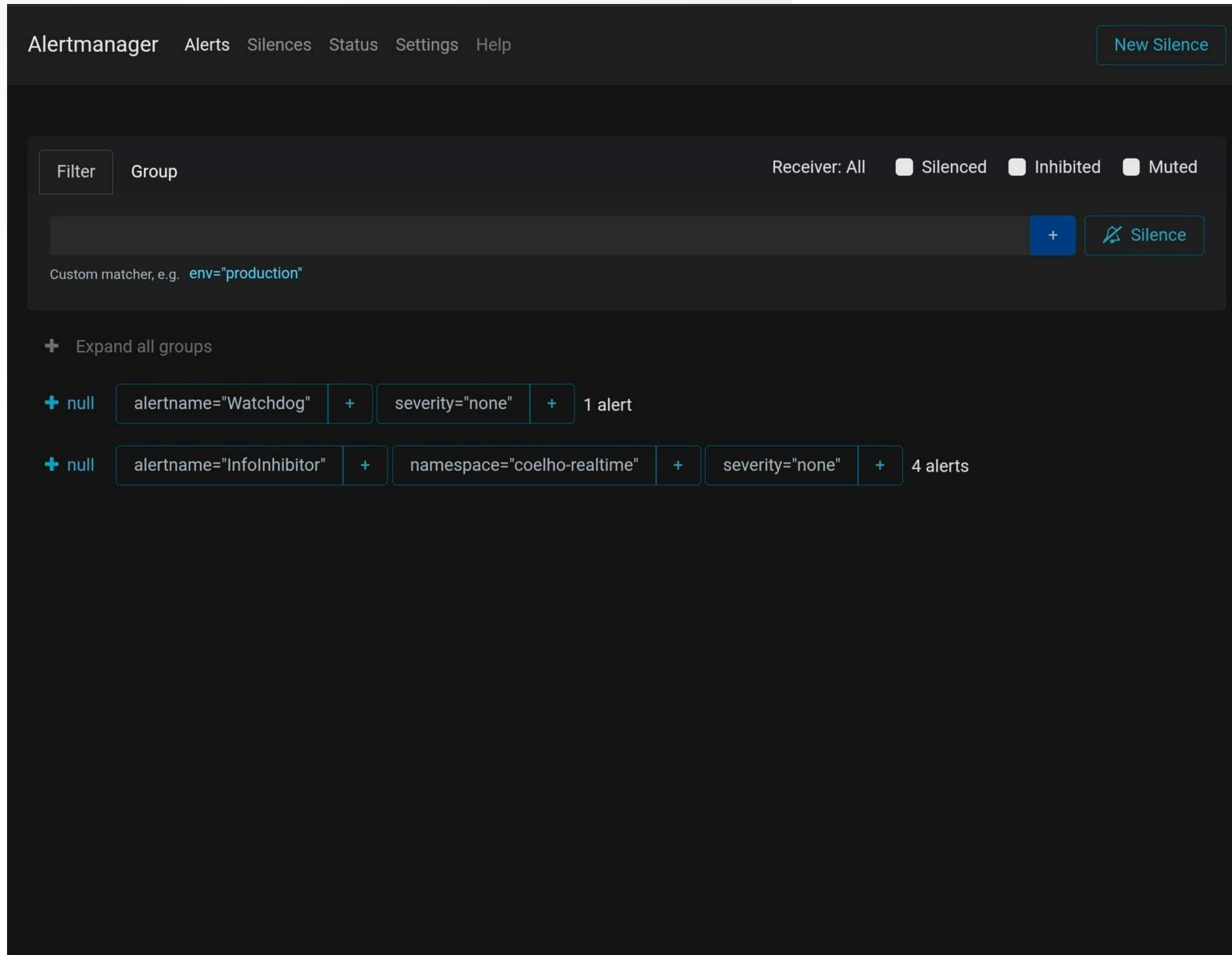
Grafana dashboards

The platform ships with 10 pre-built dashboards, automatically provisioned via Kubernetes ConfigMaps: COELHO RealTime Overview (platform-wide health), ML Pipeline (training and prediction metrics), FastAPI Detailed (API latency, throughput, errors), Kafka Producers (message rates, send latency), Kafka (broker health, partitions), Spark (master and workers), Spark Streaming (batch processing, records ingested), PostgreSQL (connections, deadlocks), Redis (memory, commands), and MinIO (storage, requests). All dashboards query Prometheus as their data source and update in real time.

MICROSERVICES - GRAFANA



MICROSERVICES - ALERTMANAGER



Alertmanager receives firing alerts from Prometheus and handles routing, grouping, deduplication, and silencing.

Alerts are grouped by alertname, namespace, and severity, with critical alerts dispatched within 10 seconds and warnings batched at 10-minute intervals.

Inhibition rules prevent cascading noise — if Kafka goes down, all downstream Kafka alerts are suppressed; if PostgreSQL is unavailable, MLflow alerts are silenced.

The routing tree supports Slack, PagerDuty, Discord, email, and webhook receivers, all pre-configured and ready to enable.

MICROSERVICES - KARMA

The screenshot displays the Karma Alertmanager dashboard with three alert panels. Each panel has a header with the alert name, namespace, severity, and a count. The first panel, 'alertname: Watchdog', has a severity of 'none' and a count of 1. The second panel, 'alertname: CPUThrottlingHigh', has a namespace of 'coelho-realtime', a severity of 'info', and a count of 1. The third panel, 'alertname: InfoInhibitor', has a namespace of 'coelho-realtime', a severity of 'none', and a count of 2. Each panel contains a description, a summary, and a list of recent alerts with their timestamps, alert states, and container names. The 'Watchdog' alert is described as ensuring the alerting pipeline is functional. The 'CPUThrottlingHigh' alert describes 73.92% CPU throttling in the 'coelho-realtime' namespace. The 'InfoInhibitor' alert is used to inhibit info alerts when a higher severity alert is present.

4

alertname: Watchdog severity: none 1

> description: This is an alert meant to ensure that the entire alerting pipeline is functional. This alert is always firing, therefore it should always be firing in Alertmanager and always fire against a receiver. There are integrations with various notification mechanisms that send a notification when this alert is not firing. For example the "DeadMansSnitch" integration in PagerDuty.

> summary: An alert that should always be firing to certify that Alertmanager is working properly.

▼ 22 minutes ago cluster: coelho-realtime

prometheus: coelho-realtime/coelho-realtime-kube-prometheus

runbook_url

alertname: CPUThrottlingHigh namespace: coelho-realtime severity: info 1

> description: 73.92% throttling of CPU in namespace coelho-realtime for container spark-worker in pod coelho-realtime-spark-worker-0 on cluster .

> summary: Processes experience elevated CPU throttling.

▼ 5 minutes ago cluster: coelho-realtime container: spark-worker

instance: 172.23.0.5:10250 pod: coelho-realtime-spark-worker-0

prometheus: coelho-realtime/coelho-realtime-kube-prometheus

service: coelho-realtime-kube-prometheus-kubelet runbook_url

alertname: InfoInhibitor namespace: coelho-realtime severity: none 2

▼ 5 minutes ago alertstate: firing container: spark-worker

pod: coelho-realtime-spark-worker-0

▼ 9 minutes ago alertstate: pending container: metrics

pod: coelho-realtime-postgresql-0

> description: This is an alert that is used to inhibit info alerts. By themselves, the info-level alerts are sometimes very noisy, but they are relevant when combined with other alerts. This alert fires whenever there's a severity="info" alert, and stops firing when another alert with a severity of 'warning' or 'critical' starts firing on the same namespace. This alert should be routed to a null receiver and configured to inhibit alerts with severity="info".

> summary: Info-level alert inhibition.

cluster: coelho-realtime instance: 172.23.0.5:10250

prometheus: coelho-realtime/coelho-realtime-kube-prometheus

service: coelho-realtime-kube-prometheus-kubelet runbook_url

Karma is a lightweight Alertmanager dashboard that provides real-time visibility into all firing, suppressed, and silenced alerts.

It connects to Alertmanager and offers label-based filtering, color-coded severity grouping, autocomplete search, and one-click silence creation – features that go beyond Alertmanager's built-in UI.

Karma enables fast alert triage during incidents by aggregating and deduplicating alerts across sources into a single overview.

“

COELHO REALTIME IS AN END-TO-END,
CLOUD-NATIVE MACHINE LEARNING
PLATFORM THAT UNIFIES REAL-TIME
STREAMING, INCREMENTAL AND BATCH
TRAINING, AND FULL OBSERVABILITY INTO A
SINGLE KUBERNETES-NATIVE
ARCHITECTURE — FROM KAFKA INGESTION
TO MODEL SERVING, FULLY AUTOMATED AND
PRODUCTION-READY.

”

LET'S WORK TOGETHER



<https://rafaelcoelho.pages.dev/>